

PROJECT TITLE

E-Commerce Data Analysis:
From Raw Data to Business Insights

SUBTITLE

A Data Analytics Project using
Python and SQL

PREPARED BY

Chaitanya Jeevankumar Sable

COURSE

- Data Analytics

TOOLS USED

- Python (Pandas, NumPy)
- SQL (MySQL Workbench)
- Data Visualization (Matplotlib)

Date – June 2026

Executive Summary

This project focuses on analyzing an e-commerce dataset to derive meaningful business insights and support data-driven decision-making. The dataset consists of multiple tables including customers, orders, products, sellers, payments, returns, and delivery logs.

The data was initially cleaned and preprocessed using Python (Pandas), where issues such as missing values, inconsistent formats, duplicate records, and incorrect data types were handled. After cleaning, the data was imported into SQL (MySQL Workbench) for further analysis.

Using SQL queries, various business problems were addressed, including revenue trends, customer behavior, product performance, seller efficiency, and delivery performance. Key metrics such as monthly revenue, customer retention, average order value, and return rates were calculated.

The analysis revealed important patterns such as higher revenue contribution from metro cities, the presence of repeat customers driving a significant portion of revenue, and certain product categories having higher return rates. Additionally, seller performance and delivery delays were identified as critical factors impacting overall business efficiency.

Based on the insights, several recommendations were proposed, including focusing on high-value customer retention, improving product quality in high-return categories, optimizing seller performance, and enhancing delivery operations.

Overall, this project demonstrates how data analysis using Python and SQL can be used to extract actionable insights and support strategic business decisions in an e-commerce environment.

Problem Statement

In today's competitive e-commerce environment, businesses generate large volumes of data from customer transactions, product sales, payments, and delivery operations. However, raw data alone does not provide value unless it is analyzed effectively to extract meaningful insights.

The primary challenge is to understand customer behaviour, identify revenue trends, evaluate product performance, and assess operational efficiency using the available data. Without proper analysis, businesses may face issues such as declining revenue, high return rates, inefficient delivery processes, and underperforming sellers.

This project aims to address these challenges by analyzing an e-commerce dataset and answering key business questions such as identifying top-performing cities and customers, understanding sales trends, evaluating product categories, and detecting operational inefficiencies.

The objective of this analysis is to transform raw data into actionable insights that can help improve revenue, enhance customer retention, optimize product strategy, and support better business decision-making.

Dataset Description

The dataset used in this project represents an e-commerce platform and consists of multiple related tables capturing different aspects of business operations. These tables include information about customers, orders, products, sellers, payments, returns, and delivery details.

Each table plays a specific role in understanding the complete business process, from customer purchase behaviour to product performance and delivery efficiency.

The tables used in this analysis are described below:

1. **Customers:** Contains customer details such as customer ID, name, city, state, gender, age, and signup date.
2. **Orders:** Stores order-level information including order ID, customer ID, order date, order status, and delivery date.
3. **Order Items:** Contains product-level details for each order, including product ID, seller ID, quantity, and item price.
4. **Products:** Includes product information such as product ID, product name, category, price, and brand.
5. **Sellers:** Contains seller details such as seller ID, seller name, city, and rating.
6. **Payments:** Stores transaction details including payment ID, order ID, payment mode, payment status, and amount.
7. **Returns:** Contains information about returned orders, including return ID, order ID, reason, and refund amount.
8. **Delivery Logs:** Includes delivery-related data such as shipped date, out-for-delivery date, and delivery time in hours.

These tables are interconnected using keys such as customer ID, order ID, product ID, and seller ID, enabling comprehensive analysis across different dimensions of the business.

Cleaning and basic insights

1) Customers Table

```
Customers Table Cleaning

print('-----\n1) Duplicate customer_id: ',customers['customer_id'].duplicated().sum())

## Date column
customers['signup_date'] = pd.to_datetime(customers['signup_date'], format = 'mixed', errors = 'coerce').dt.strftime('%Y-%m-%d')

## Age column
customers['age'] = customers['age'].round().astype('Int64')

## Full name column (null handling) / strip / regex / title
customers['full_name'].fillna('Unknown', inplace = True)
customers['full_name'] = customers['full_name'].str.strip().str.title()
customers['full_name'] = customers['full_name'].str.replace(r'\s+', ' ', regex= True)

## Age column (null handling)/ We checked the skewness of age column and it was found to be right skewed.
# So we used median to fill null values.
print('-----\n2) Skewness of age column: ',customers['age'].skew())
customers['age'] = customers['age'].fillna(customers['age'].median())

## State and city column
print('-----\nUnique No. of states: ',customers['state'].nunique())
print('-----\nUnique No. of cities: ',customers['city'].nunique())

print('-----\nUnique states: ',customers['state'].unique())
print('-----\nUnique cities: ',customers['city'].unique())
```

```
## Replace/ Strip/ Title of States and Cities
customers['state'] = customers['state'].replace(
    {'MH':'Maharashtra',
     'GJ':'Gujarat',
     'KA':'Karnataka',
     'UP':'Uttar Pradesh',
     'TS':'Tamil Nadu',
     'WB':'West Bengal',
     'RJ':'Rajasthan',
     'DL':'Delhi'}
).str.strip().str.title()

customers['city'] = customers['city'].str.strip().str.title()

## Gender column
print('-----\n4) Gender',customers['gender'].unique())
customers['gender'] = customers['gender'].replace({'M':'Male','F':'Female'})
```

```
1) Duplicate customer_id: 0
-----
2) Skewness of age column: -0.29097833717120103
-----
Unique No. of states: 12
-----
Unique No. of cities: 20
-----
Unique states: ['MH' 'Maharashtra' 'Tamil Nadu' 'GJ' 'Karnataka' 'KA' 'TS' 'WB' 'RJ'
'Delhi' 'UP' 'DL']
-----
Unique cities: ['Kolkata' 'Bengaluru' 'chennai' 'jaipur' 'Ahmedabad' 'Lucknow' 'Jaipur'
'Mumbai' 'Chennai' 'pune' 'Hyderabad' 'Pune' 'mumbai' 'Delhi' 'bengaluru'
'delhi' 'lucknow' 'kolkata' 'ahmedabad' 'hyderabad']
-----
```

```
print(f'-----\nNull values in column\n-----\n{customers.isnull().sum()}\n-----')
print('Number of rows', customers.shape[0])
print('Number of columns', customers.shape[1])
print('Unique No. of customers: ',customers['customer_id'].nunique())

-----
Null values in column
-----
customer_id    0
full_name      0
city           0
state          0
signup_date    0
gender         0
age            0
dtype: int64
-----
Number of rows 50000
Number of columns 7
Unique No. of customers: 50000
```

Data Cleaning Overview

The customer dataset was examined and cleaned to ensure consistency, accuracy, and usability for further analysis. Initially, a null value check was performed across all columns, and it was observed that there were no missing values in the dataset.

A duplicate check was conducted on the `customer_id` column, confirming that all records are unique and no duplicate entries exist. This ensures data integrity at the customer level.

The `signup_date` column was standardized by converting it into a consistent date format. This step helps in accurate time-based analysis and avoids issues caused by mixed date formats.

The age column was processed by rounding off decimal values and converting them into integer format for better readability and consistency. Additionally, statistical analysis such as skewness was checked to understand the distribution of age values.

Text columns such as `full_name`, `city`, and `state` were cleaned by removing extra spaces and converting text into proper title case. Inconsistent state abbreviations were replaced with their full names to maintain uniformity across the dataset.

The gender column values were also standardized by converting abbreviated forms (M/F) into full forms (Male/Female) for better clarity.

Summary

Overall, the dataset was found to be largely clean, with no missing or duplicate values. Minor inconsistencies in text formatting and data representation were corrected to improve data quality. These cleaning steps ensure that the dataset is reliable and ready for further analysis and visualization.

2) Orders Table

```
Orders Table Cleaning

## Summary of orders table
print(f'-----\nNull values in column\n-----\n{orders.isnull().sum()}\n-----')
print(f'1) Duplicate order_id: {orders["order_id"].duplicated().sum()} \n\tAnd number of orders : ',orders["order_id"].nunique())
print(f'3) Number of rows', orders.shape[0])
print(f'4) Number of columns', orders.shape[1])

## Strip/title on order_id, customer_id, order_status
text_col = orders.select_dtypes(include=['object']).columns
text_col = [col for col in text_col if col not in ['order_date','delivery_date']]
for col in text_col:
    orders[col] = orders[col].str.strip().str.title()

## Order date column
orders['order_date'] = pd.to_datetime(orders['order_date'], format = 'mixed', errors = 'coerce').dt.strftime('%Y-%m-%d')

## order status count
print(f'-----\nOrder Status\n-----\n{orders.groupby("order_status")["order_id"].count()}\n-----')

## Number of null in delivery_date according to order_status
print(
    "-----\n"
    "Number of null in delivery_date according to order_status\n"
    "-----\n"
    f"{orders.groupby('order_status')['delivery_date']\n"
    ".apply(lambda x: x.isnull().sum())}\n"
    "-----"
)
```

```
## Create new column is_delivered/Returned
orders["is_delivered/Returned"] = orders["order_status"].isin(["Delivered", "Returned"]).astype(int)

## Checking if all cancelled and shipped orders have delivery_date
print('1)',orders[orders["order_status"] == "Cancelled"]["delivery_date"].isnull().all())
print('1)',orders[orders["order_status"] == "Shipped"]["delivery_date"].isnull().all())

## Convert delivery date to date format
orders['delivery_date'] = pd.to_datetime(orders['delivery_date'], format = 'mixed', errors = 'coerce').dt.strftime('%Y-%m-%d')

orders.head()
```

```
Null values in column
-----
order_id      0
customer_id   0
order_date    0
order_status  0
delivery_date 39965
dtype: int64
-----
1) Duplicate order_id: 0
   And number of orders : 200000
3) Number of rows 200000
4) Number of columns 5
-----
Order Status
-----
order_status
Cancelled    19929
Delivered    144082
Returned     15953
Shipped       20036
Name: order_id, dtype: int64
-----
...
Name: delivery_date, dtype: int64
-----
1) True
1) True
```

Orders Data Cleaning Overview

The orders dataset was analyzed and cleaned to ensure data consistency and reliability. Initially, a null value check was performed, which revealed a significant number of missing values in the delivery_date column, while all other columns were complete.

A duplicate check on the order_id column confirmed that all records are unique, ensuring that there are no repeated transactions in the dataset.

Text-based columns such as `order_status` were cleaned by removing extra spaces and converting values into a consistent title case format. This helps in avoiding mismatches during grouping and analysis.

The `order_date` and `delivery_date` columns were standardized by converting them into a uniform date format. This step ensures proper time-based analysis and eliminates inconsistencies due to mixed date formats.

Further analysis was performed to understand the distribution of orders across different statuses. It was observed that delivery-related information is missing primarily for orders that are either cancelled or still in the shipping stage.

To validate this, a check was performed to confirm that all cancelled and shipped orders do not have a delivery date, which aligns with real-world business logic.

Additionally, a new column was created to categorize whether an order was successfully completed (Delivered/Returned) or not. This transformation simplifies further analysis and reporting.

Summary

The dataset was largely clean with no duplicate records. The only major issue identified was missing delivery dates, which were logically consistent with order statuses such as cancelled or shipped. Necessary transformations, including text standardization and date formatting, were applied to ensure the dataset is structured and ready for further analysis.

3) Products Table

```
Products Table Cleaning

## Summary of products table
print(products.info(), "\n-----")

## Null values in products table
print('NULL VALUES:\n----- \n', products.isnull().sum(), "\n-----")

## Duplicate values in products table
print('DUPLICATE VALUES:', products.duplicated().sum(), "\n-----")

## Unique Brand name
print('BRANDS:\n----- \n', products['brand'].unique(), "\n-----")

## Strip/Title on product_id, brand, product_name
text_col = products.select_dtypes(include = 'object').columns

excluded = ['price', 'product_id']

for col in text_col:
    if col not in excluded:
        products[col] = products[col].str.strip().str.title()

## Price column
products['price'] = products['price'].round().astype('int')

## Brand column
products['brand'] = products['product_name'].str.split(' ', n = 1).str[0]

products.head()
```


Products Data Cleaning Overview

The products dataset was thoroughly examined to ensure accuracy, consistency, and readiness for further analysis. Initially, a null value check was performed across all columns, and it was observed that there were no missing values present in the dataset. This indicates that the dataset was complete and did not require any imputation techniques.

A duplicate check was also conducted, which confirmed that there were no duplicate records in the dataset. This ensures that each product entry is unique and reliable for analysis.

Further inspection of the brand column revealed inconsistencies in naming conventions, such as differences in letter casing (e.g., "nike", "NIKE", "Nike"). To address this, text cleaning techniques were applied, including removing extra spaces and converting all text values into a standardized title case format. This step helps in avoiding duplication issues during grouping and aggregation.

Similarly, other text-based columns such as product_name and category were cleaned using string operations like stripping unnecessary spaces and applying proper capitalization. This improves readability and ensures uniformity across the dataset.

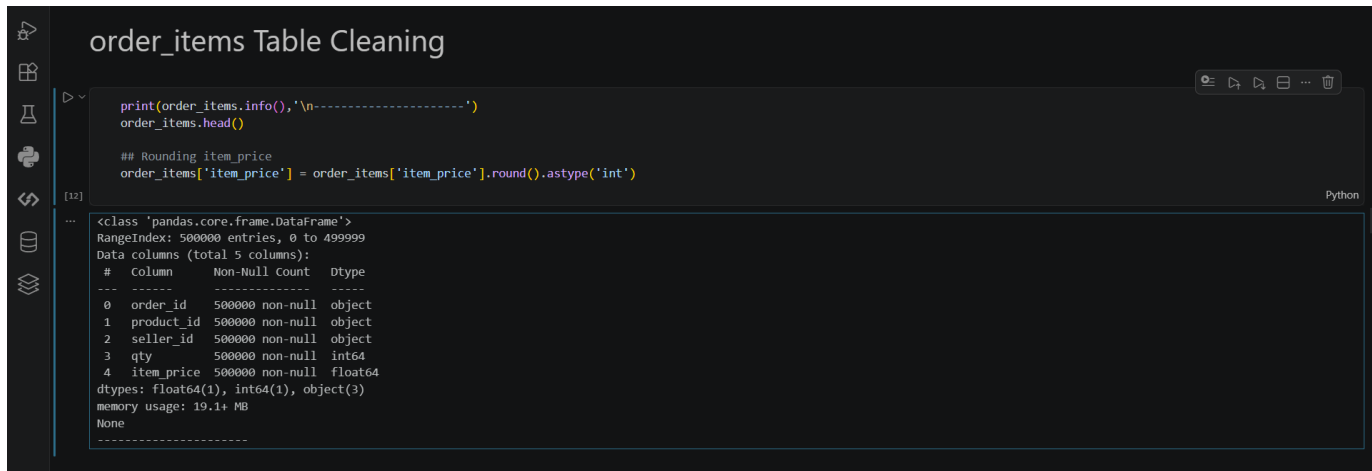
The price column was processed by rounding off decimal values and converting them into integer format. This step ensures consistency in numerical representation and simplifies further calculations.

Additionally, the brand name was derived from the product_name column using string splitting techniques. This was done to ensure consistency in brand identification and to handle cases where the brand information may not be properly structured.

Summary

Overall, the products dataset was found to be well-structured with no missing or duplicate values. Minor inconsistencies in text formatting and brand naming were corrected. These cleaning steps improved data uniformity and ensured that the dataset is reliable and ready for further analysis and visualization.

4) Order Items



The screenshot shows a Jupyter Notebook interface with a dark theme. The notebook title is "order_items Table Cleaning". The code cell contains the following Python code:

```
print(order_items.info(),'\n-----')
order_items.head()

## Rounding item_price
order_items['item_price'] = order_items['item_price'].round().astype('int')
```

The output of the code is displayed below the cell:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500000 entries, 0 to 499999
Data columns (total 5 columns):
#   column      Non-Null Count  Dtype
---  -
0   order_id    500000 non-null         object
1   product_id  500000 non-null         object
2   seller_id   500000 non-null         object
3   qty         500000 non-null         int64
4   item_price  500000 non-null         float64
dtypes: float64(1), int64(1), object(3)
memory usage: 19.1+ MB
None
```

Order Items Data Cleaning Overview

The `order_items` dataset was analyzed to ensure data consistency and accuracy. An initial inspection using the dataset information confirmed that there were no missing values across any of the columns, indicating that the dataset was complete.

The dataset contains transactional-level data with attributes such as order ID, product ID, seller ID, quantity, and item price. Each column was checked for appropriate data types to ensure smooth processing and analysis.

The `item_price` column was cleaned by rounding off decimal values and converting them into integer format. This step helps in maintaining consistency in numerical representation and simplifies further calculations such as total revenue or average pricing.

Since all columns were found to be complete and properly structured, no major cleaning operations such as null handling or duplicate removal were required.

Summary

The `order_items` dataset was largely clean and well-structured. Only minor transformation was required in the form of price standardization. The dataset is now ready for further analysis, including revenue calculation and product-level insights.

5) Payment Table

```
Payment Table Cleaning

print(payments.info(),'\n-----')

## Payment mode
print('-----\nPAYMENT MODES\n-----\n',payments['payment_mode'].unique())

## Payment status
print('PAYMENT STATUS\n-----\n',payments['payment_status'].unique())

## Duplicate payment_id
print('NUMBER OD DUPLICATES IN PAYMENT ID\n-----\n',payments['payment_id'].duplicated().sum())

## Amount column
payments['amount'] = payments['amount'].round().astype('int')

payments.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 5 columns):
 #   Column          Non-Null count  Dtype
---  -
 0   order_id        200000 non-null object
 1   payment_id      200000 non-null object
 2   payment_mode    200000 non-null object
 3   payment_status  200000 non-null object
 4   amount          200000 non-null float64
dtypes: float64(1), object(4)
memory usage: 7.6+ MB
None
-----
PAYMENT MODES
-----
['NetBanking' 'Card' 'UPI' 'COD']
PAYMENT STATUS
-----
['Success' 'Failed' 'Refunded']
NUMBER OD DUPLICATES IN PAYMENT ID
-----
0

order_id  payment_id  payment_mode  payment_status  amount
0  O400000      PAY0      NetBanking      Success      37023
1  O400001      PAY1        Card      Success      12485
2  O400002      PAY2        UPI      Success      110876
3  O400003      PAY3        COD      Success      106650
4  O400004      PAY4        Card      Success      104723
```

Payment Data Cleaning Overview

The payments dataset was analyzed to ensure data quality and consistency. An initial inspection confirmed that there were no missing values across any of the columns, indicating that the dataset was complete and reliable.

A duplicate check was performed on the payment_id column, and no duplicate records were found. This ensures that each payment transaction is uniquely recorded.

Further analysis was carried out on categorical columns such as payment_mode and payment_status. It was observed that the values were already well-structured, with clearly defined categories like Net Banking, Card, UPI, and Cash on Delivery (COD) for payment modes, and Success, Failed, and Refunded for payment status.

The amount column was cleaned by rounding off decimal values and converting them into integer format. This standardization helps in maintaining consistency and simplifies financial analysis.

Summary

The payments dataset was found to be clean and well-structured with no missing or duplicate values. Minor formatting improvements were applied to numerical data, ensuring that the dataset is ready for accurate financial analysis and reporting.

6) Returns Table

```
returns

## Summary of returns table
print(returns.info(),'\n-----')

## Duplicate return id
print('DUPLICATE RETURN ID\n-----\n',returns['return_id'].duplicated().sum())

## Unique reasons
print('UNIQUE REASONS\n-----\n',returns['reason'].unique())

## Refund amount
returns['refund_amount'] = returns['refund_amount'].round().astype('int')

returns.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15953 entries, 0 to 15952
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype  
---  --
 0   order_id        15953 non-null  object 
 1   return_id       15953 non-null  object 
 2   reason          15953 non-null  object 
 3   refund_amount   15953 non-null  float64
dtypes: float64(1), object(3)
memory usage: 498.7+ KB
None
-----
DUPLICATE RETURN ID
-----
0
UNIQUE REASONS
-----
['Damaged' 'Wrong Product' 'Late Delivery' 'Quality Issue']

...

```

	order_id	return_id	reason	refund_amount
0	O400009	R0	Damaged	49236
1	O400024	R1	Wrong Product	15696
2	O400087	R2	Late Delivery	4882
3	O400106	R3	Late Delivery	55956
4	O400115	R4	Wrong Product	43842

Returns Data Cleaning Overview

The returns dataset was analyzed to ensure data accuracy and consistency. An initial inspection confirmed that there were no missing values across any of the columns, indicating that the dataset was complete and reliable.

A duplicate check was performed on the return_id column, and no duplicate records were found. This ensures that each return transaction is uniquely recorded.

Further analysis of the reason column showed well-defined categories such as Damaged, Wrong Product, Late Delivery, and Quality Issue. These values were already consistent and required no additional standardization.

The refund_amount column was processed by rounding off decimal values and converting them into integer format. This helps in maintaining consistency in financial data and simplifies further analysis.

Summary

The returns dataset was found to be clean and well-structured with no missing or duplicate values. Only minor numerical formatting was applied, making the dataset ready for return analysis and refund-related insights.

7) Sellers Table

```
Sellers

## Summary of sellers table
sellers.info()

## City column
sellers['city'] = sellers['city'].str.strip().str.title().str.replace(r'\s+', '', regex=True)

## Number of unique sellers
print('-----\nNumber of unique sellers :', sellers['seller_name'].nunique(), '\n-----')

## Unique Cities
print('Unique Cities :', sellers['city'].unique())

## Rating
sellers['rating'] = sellers['rating'].fillna(sellers['rating'].mean())

sellers.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 4 columns):
 #   Column        Non-Null Count  Dtype  
---  -
 0   seller_id     2000 non-null   object  
 1   seller_name   2000 non-null   object  
 2   city          2000 non-null   object  
 3   rating        1027 non-null   float64  
dtypes: float64(1), object(3)
memory usage: 62.6+ KB

Number of unique sellers : 2000
-----
Unique Cities : ['Hyderabad' 'Bengaluru' 'Chennai' 'Lucknow' 'Delhi' 'Jaipur' 'Ahmedabad'
 'Pune' 'Kolkata' 'Mumbai']

...
  seller_id  seller_name  city  rating
0  S300000    Seller_0  Hyderabad  3.715969
1  S300001    Seller_1  Bengaluru  3.600000
2  S300002    Seller_2  Chennai  3.715969
3  S300003    Seller_3  Lucknow  4.500000
4  S300004    Seller_4  Delhi  3.715969
```

Sellers Data Cleaning Overview

The sellers dataset was analyzed to ensure consistency and data quality. An initial inspection showed that most columns were complete, except for the rating column, which contained missing values.

To handle this, the missing values in the rating column were replaced using the mean value of the existing ratings. This approach helps in maintaining the overall distribution of the data without introducing bias.

The city column was cleaned by removing extra spaces and converting all values into a standardized title case format. This ensures uniformity and avoids duplication issues caused by inconsistent text formatting.

Further analysis confirmed that all seller records are unique, with a total of 2000 unique sellers in the dataset. Additionally, the dataset includes sellers from multiple cities, providing a diverse geographical distribution.

Summary

Minor cleaning was performed for missing ratings and text formatting. The dataset is now clean and ready for analysis.

8) Delivery Logs Table



```
delivery_logs.head()

## shipped date
delivery_logs['shipped_date'] = pd.to_datetime(delivery_logs['shipped_date'], format = 'mixed', errors = 'coerce').dt.strftime('%Y-%m-%d')

## out_for_delivery
delivery_logs['out_for_delivery'] = pd.to_datetime(delivery_logs['out_for_delivery'], format = 'mixed', errors = 'coerce').dt.strftime('%Y-%m-%d')
```

```
delivery_logs.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 160035 entries, 0 to 160034
Data columns (total 4 columns):
#   Column              Non-Null Count  Dtype  
---  -
0   order_id             160035 non-null  object 
1   shipped_date         160035 non-null  object 
2   out_for_delivery     160035 non-null  object 
3   delivered_time_hours 160035 non-null  int64  
dtypes: int64(1), object(3)
memory usage: 4.9+ MB
```

Delivery Logs Data Cleaning

The delivery logs dataset was cleaned by converting the shipped_date and out_for_delivery columns into a standard date format to ensure consistency. All columns were found to have no missing values, indicating a complete dataset.

Summary

Minimal cleaning was required. Date columns were standardized, and the dataset is ready for delivery performance analysis.

Data export and import into MySQL workbench

After completing the data cleaning process in Python, the cleaned datasets were exported as CSV files and imported into MySQL Workbench for further analysis.

Each dataset was loaded as a separate table in the database, including customers, orders, order items, products, sellers, payments, returns, and delivery logs.

During the import process, appropriate data types were assigned to each column. For example, date columns such as order_date and delivery_date were stored using the DATE data type, while numerical columns such as amount and quantity were stored as numeric types.

Primary keys and relationships between tables were maintained using common fields such as customer_id, order_id, product_id, and seller_id. This allowed efficient joining of tables for analytical queries.

Once the data was successfully loaded into the database, SQL queries were used to extract insights and answer key business questions.

Exporting to CSV

```
customers.to_csv(r'C:\Users\Chaitanya\Downloads\ecommerce_large_dataset\customers_c.csv', index = False)
orders.to_csv(r'C:\Users\Chaitanya\Downloads\ecommerce_large_dataset\orders_c.csv', index = False)
products.to_csv(r'C:\Users\Chaitanya\Downloads\ecommerce_large_dataset\products_c.csv', index = False)
order_items.to_csv(r'C:\Users\Chaitanya\Downloads\ecommerce_large_dataset\order_items_c.csv', index = False)
payments.to_csv(r'C:\Users\Chaitanya\Downloads\ecommerce_large_dataset\payments_c.csv', index = False)
returns.to_csv(r'C:\Users\Chaitanya\Downloads\ecommerce_large_dataset\returns_c.csv', index = False)
sellers.to_csv(r'C:\Users\Chaitanya\Downloads\ecommerce_large_dataset\sellers_c.csv', index = False)
delivery_logs.to_csv(r'C:\Users\Chaitanya\Downloads\ecommerce_large_dataset\delivery_logs_c.csv', index = False)
```

[18]

Python

Importing to MySQL with the help of SQLAlchemy

```
from sqlalchemy import create_engine

# CSV read
df = pd.read_csv(r'C:\Users\Chaitanya\OneDrive\Desktop\cleaned dataset\customers_c.csv')

# MySQL connection
engine = create_engine("mysql+pymysql://root:Csm%401011@localhost/projects")

# Direct table create + data insert
df.to_sql(
    name="customers_c",
    con=engine,
    if_exists="replace", # old table replace
    index=False,
    chunksize=1000,
    method="multi"
)

print("Data Imported Successfully")
```

[28]

Python

... Data Imported Successfully

My Sql Query Solving

- Tables

```
1
2 • SELECT * FROM customers_c;
3 • SELECT * FROM orders_c;
4 • SELECT * FROM order_items_c;
5 • SELECT * FROM products_c;
6 • SELECT * FROM payments_c;
7 • SELECT * FROM delivery_logs_c;
8 • SELECT * FROM returns_c;
9 • SELECT * FROM sellers_c;
```

1) What is the monthly revenue trend of last 12 months?

```
1  # What is the monthly revenue trend of last 12 months?
2
3 • SELECT
4  DATE_FORMAT(STR_TO_DATE(oc.order_date, '%Y-%m-%d'), '%Y-%m') AS order_month,
5  FORMAT(SUM(pc.amount),0) AS revenue
6      FROM orders_c oc
7      INNER JOIN payments_c pc
8          ON oc.order_id = pc.order_id
9  WHERE oc.order_status = 'Delivered'
10      AND pc.payment_status = 'Success'
11      AND str_to_date(oc.order_date, '%Y-%m-%d') >= (
12          SELECT
13              MAX(STR_TO_DATE(order_date, '%Y-%m-%d'))
14          FROM
15              orders_c
16      ) - INTERVAL 12 MONTH
17  GROUP BY order_month
18  ORDER BY order_month;
```

Result Grid		
Filter Rows:		
Export: 		
	order_month	revenue
▶	2024-04	8,697,457
	2024-05	254,965,824
	2024-06	244,447,490
	2024-07	259,515,938
	2024-08	261,526,298
	2024-09	249,161,167
	2024-10	259,704,705
	2024-11	255,723,811
	2024-12	259,844,980
	2025-01	258,291,282
	2025-02	241,078,855
	2025-03	252,612,146
	2025-04	254,183,924

2) What is Month-over-Month Growth %

```
1      # What is Month-over-Month Growth %
2
3      SELECT *,CONCAT(ROUND(((x.current_month_revenue-x.previous_month_revenue)/x.previous_month_revenue),2),' ','%') AS month_over_month_growth FROM
4      (SELECT
5          DATE_FORMAT(STR_TO_DATE(oc.order_date, '%Y-%m-%d'), '%Y-%m') as order_month,
6          SUM(pc.amount) AS current_month_revenue,
7          LAG(SUM(pc.amount)) OVER(ORDER BY DATE_FORMAT(STR_TO_DATE(oc.order_date, '%Y-%m-%d'), '%Y-%m')) AS previous_month_revenue
8      FROM orders_c oc
9      INNER JOIN payments_c pc
10     ON oc.order_id = pc.order_id
11     WHERE oc.order_status = 'Delivered'
12         AND pc.payment_status = 'Success'
13         AND STR_TO_DATE(oc.order_date, '%Y-%m-%d') >=
14             (SELECT MAX(STR_TO_DATE(order_date, '%Y-%m-%d' ))
15              FROM orders_c) - INTERVAL 13 MONTH
16     GROUP BY order_month
17     LIMIT 100
18     OFFSET 1)
19     x;
```

Result Grid				
Filter Rows:				
Export:				
Wrap Cell Content:				
	order_month	current_month_revenue	previous_month_revenue	month_over_month_growth
▶	2024-04	255040260	18654271	12.67 %
	2024-05	254965824	255040260	0.00 %
	2024-06	244447490	254965824	-0.04 %
	2024-07	259515938	244447490	0.06 %
	2024-08	261526298	259515938	0.01 %
	2024-09	249161167	261526298	-0.05 %
	2024-10	259704705	249161167	0.04 %
	2024-11	255723811	259704705	-0.02 %
	2024-12	259844980	255723811	0.02 %
	2025-01	258291282	259844980	-0.01 %
	2025-02	241078855	258291282	-0.07 %
	2025-03	252612146	241078855	0.05 %
	2025-04	254183924	252612146	0.01 %

3) Top 10 Highest Revenue Generating Cities?

```
1  # Top 10 Highest Revenue Generating Cities?
2
3  • SELECT
4  city,
5  FORMAT(SUM(pc.amount),0) AS total_revenue
6  FROM
7  customers_c cc
8  INNER JOIN orders_c oc
9  ON cc.customer_id = oc.customer_id
10 INNER JOIN payments_c pc
11 ON oc.order_id = pc.order_id
12 WHERE oc.order_status = 'Delivered' AND pc.payment_status = 'success'
13 GROUP BY city
14 ORDER BY total_revenue DESC
15 LIMIT 10;
```

Result Grid	Filter Rows:	Export:
city	total_revenue	
Chennai	724,035,916	
Ahmedabad	715,710,236	
Lucknow	714,103,587	
Mumbai	713,275,796	
Kolkata	712,170,758	
Hyderabad	710,853,139	
Pune	710,680,328	
Delhi	703,418,066	
Jaipur	699,998,353	
Bengaluru	692,055,156	

4) Average Order Value (AOV) City-wise?

```
1  # Average Order Value (AOV) City-wise?
2
3  • SELECT
4    x.city,
5    x.total_revenue,
6    round((x.total_revenue / x.count_of_orders), 2) AS AOV_Average_order_value
7  FROM
8    (SELECT
9      city,
10     SUM(pc.amount) AS total_revenue,
11     COUNT(DISTINCT oc.order_id) count_of_orders
12   FROM
13     customers_c cc
14     INNER JOIN orders_c oc ON cc.customer_id = oc.customer_id
15     INNER JOIN payments_c pc ON oc.order_id = pc.order_id
16   WHERE oc.order_status = 'Delivered' AND pc.payment_status = 'success'
17   GROUP BY city
18   ORDER BY total_revenue DESC) x;
```

	city	total_revenue	AOV_Average_order_value
►	Chennai	724035916	60051.08
	Ahmedabad	715710236	60042.81
	Lucknow	714103587	59933.16
	Mumbai	713275796	60263.25
	Kolkata	712170758	60348.34
	Hyderabad	710853139	59540.43
	Pune	710680328	60242.46
	Delhi	703418066	60327.45
	Jaipur	699998353	60443.69
	Bengaluru	692055156	60136.87

5) Top 20 Customers by Lifetime Revenue

```
1  # Top 20 Customers by Lifetime Revenue
2
3  • SELECT
4  row_number() over (ORDER BY sum(pc.amount) DESC) AS Rank_,
5  cc.customer_id,
6  cc.full_name,
7  FORMAT(SUM(pc.amount),0) AS Lifetime_revenue
8  FROM
9      customers_c cc
10     INNER JOIN orders_c oc ON cc.customer_id = oc.customer_id
11     INNER JOIN payments_c pc ON oc.order_id = pc.order_id
12     WHERE oc.order_status = 'Delivered' AND pc.payment_status = 'Success'
13 GROUP BY cc.customer_id, cc.full_name
14 LIMIT 20;
```

	Rank_	customer_id	full_name	Lifetime_revenue
►	1	C103217	Vikas Verma	781,235
	2	C137646	Amit Iyer	712,511
	3	C108040	Arjun Sharma	705,196
	4	C127355	Priya Reddy	704,418
	5	C139564	Sneha Verma	698,483
	6	C117634	Priya Verma	689,472
	7	C110855	Anjali Gupta	679,230
	8	C144949	Neha Reddy	674,674
	9	C114880	Unknown	670,359
	10	C119573	Amit Verma	661,462
	11	C139551	Rahul Gupta	658,672
	12	C115470	Sneha Patil	651,586
	13	C110340	Rahul Patil	647,811
	14	C124192	Anjali Verma	644,870
	15	C104406	Rohit Jain	644,268
	16	C100796	Vikas Das	638,087
	17	C106243	Sneha Singh	637,938
	18	C100577	Vikas Singh	637,652
	19	C118217	Arjun Sharma	636,928
	20	C101330	Anjali Das	633,460

6) Weekend vs Weekday sales comparison

```
1  # Weekend vs Weekday sales comparison
2
3  • SELECT
4      CASE
5          WHEN DAYNAME(oc.order_date) IN ('Saturday', 'Sunday')
6              THEN 'weekend'
7              ELSE 'weekday'
8      END AS day_name,
9      FORMAT(SUM(pc.amount),0) AS Revenue,
10     ROUND(SUM(pc.amount) * 100 /
11         (SELECT SUM(pc2.amount)
12         FROM orders_c oc2
13         INNER JOIN payments_c pc2
14         ON oc2.order_id = pc2.order_id
15         WHERE oc2.order_status = 'Delivered'
16         AND pc2.payment_status = 'Success'),2) AS percentage_contribution
17 FROM orders_c oc
18     INNER JOIN payments_c pc
19     ON oc.order_id = pc.order_id
20     WHERE oc.order_status = 'delivered' AND pc.payment_status = 'success'
21     GROUP BY day_name;
22
```

	day_name	Revenue	percentage_contribution
▶	weekday	5,073,789,958	71.50
	weekend	2,022,511,377	28.50

7) Net Revenue excluding cancelled orders

```
1  # Net Revenue excluding cancelled ordrs
2
3  • SELECT FORMAT(SUM(pc.amount),0) AS Net_revenue
4      FROM orders_c oc
5      INNER JOIN payments_c pc
6      ON oc.order_id = pc.order_id
7      WHERE order_status != 'Cancelled'
8      AND pc.payment_status = 'success';
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Net_revenue			
▶ 8,879,294,203			

8) Highest revenue generating category

```
1  # Highest revenue categories
2
3  • SELECT proc.category,FORMAT(SUM(oic.qty*oic.item_price),0) AS total_revenue
4  FROM order_items_c oic
5  INNER JOIN products_c proc
6  ON oic.product_id = proc.product_id
7  GROUP BY category
8  ORDER BY SUM(oic.qty*oic.item_price)
9  DESC LIMIT 1;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	category	total_revenue		
▶	Mobile	16,589,789,813		

9) Repeat customers vs one-time buyers count.

```
1  # Repeat customers vs one-time buyers count.
2
3  • select
4  x.repeat_customers,
5  y.one_time_customers from
6  (select count(x1.repeat_cust) as repeat_customers
7  from
8  (select count(customer_id) as repeat_cust
9  from orders_c
10  group by customer_id
11  having count(customer_id) > 1) x1) x,
12  (select count(y1.one_time) as one_time_customers from
13  (select count(customer_id) as one_time
14  from orders_c
15  group by customer_id
16  having count(customer_id) = 1) y1) y;
17
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
	repeat_customers	one_time_customers	
▶	45405	3681	

10) What is the Customer retention rate?

```
1  # What is the Customer retention rate?
2
3  • select concat(round((x.repeat_customers/y.Total_customers*100),2), ' ', '%') as customer_retention_rate
4  from
5      (select count(x1.repeat_cust) as repeat_customers
6       from
7           (select count(order_id) as repeat_cust
8            from orders_c
9            group by customer_id
10           having count(order_id) > 1) x1
11      )x,
12      (select count(distinct customer_id) as Total_customers from orders_c)y;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	customer_retention_rate			
▶	92.50 %			

11) Cohort analysis by first purchase month

```
1  # Cohort analysis by first purchase month
2
3  • select x.first_purchase_month, x.month_number, count(x.customer_id) as First_time_purchase_count
4  from
5      (select customer_id, monthname(x1.first_purchase) as first_purchase_month, month(x1.first_purchase) as month_number
6       from
7           (select customer_id, min(order_date) as first_purchase
8            from orders_c
9            group by customer_id) x1
10      ) x
11  group by x.first_purchase_month, x.month_number
12  order by x.month_number asc;
```

	first_purchase_month	month_number	First_time_purchase_count
▶	January	1	8193
	February	2	6476
	March	3	6185
	April	4	5285
	May	5	4553
	June	6	3725
	July	7	3443
	August	8	3088
	September	9	2437
	October	10	2203
	November	11	1886
	December	12	1612

12) Purchase frequency top customers

```
1  # Purchase frequency top customers
2
3  • select oc.customer_id, cc.full_name,
4      sum(pc.amount) as spending,
5      count(distinct oc.order_id) as purchase_frequency
6  from customers_c cc
7  inner join orders_c oc on cc.customer_id = oc.customer_id
8  inner join payments_c pc on oc.order_id = pc.order_id
9  where oc.order_status = 'Delivered' and pc.payment_status = 'success'
10 group by oc.customer_id, cc.full_name
11 order by purchase_frequency desc
12 limit 10;
```

	customer_id	full_name	spending	purchase_frequency
▶	C128719	Sneha Khan	502864	11
	C128708	Priya Reddy	601751	11
	C144949	Neha Reddy	674674	10
	C145114	Arjun Jain	337895	10
	C137875	Amit Verma	532503	10
	C140802	Vikas Khan	475253	10
	C141403	Rahul Gupta	548838	10
	C112286	Neha Reddy	438862	10
	C101535	Rahul Singh	579635	10
	C101734	Arjun Gupta	527620	10

13) Count of 90+ day inactive customers

```
1  # Count of 90+ day inactive customers
2
3  • select count(x.customer_id) as inactive_since_90Days
4  from
5  (select oc.customer_id, count(oc.order_id) as total_orders, max(oc.order_date) as last_purchase_date
6   from orders_c oc
7   inner join payments_c pc
8   on oc.order_id = pc.order_id
9   where oc.order_status = 'Delivered' and pc.payment_status = 'success'
10  group by oc.customer_id
11  having max(oc.order_date) < current_date() - interval 90 day) x
```

	inactive_since_90Days
▶	45301

14) Avg days between first & second purchase

```
1  # Avg days between first & second purchase
2
3  • select avg(h.days_gap) as avg_days_between_first_second_purchase
4  from
5  (select a.customer_id, a.order_date as first_date, b.order_date as second_date, datediff(b.order_date, a.order_date) as days_gap
6  from
7  (select *
8  from
9  (select oc.customer_id, oc.order_date, ROW_NUMBER() over(partition by oc.customer_id order by oc.order_date) as rownum
10  from orders_c oc
11  inner join payments_c pc
12  on oc.order_id = pc.order_id
13  where (oc.order_status = 'Delivered' and pc.payment_status = 'success')) x
14  where x.rownum = 1) a
15  inner join
16  (select *
17  from
18  (select oc.customer_id, oc.order_date, ROW_NUMBER() over(partition by oc.customer_id order by oc.order_date) as rownum
19  from orders_c oc
20  inner join payments_c pc
21  on oc.order_id = pc.order_id
22  where (oc.order_status = 'Delivered' and pc.payment_status = 'success')) x
23  where x.rownum = 2) b
24  on a.customer_id = b.customer_id) h;
```

15) RFM Segmentation

```
1  # RFM Segmentation.
2
3  • select x.customer_id, x.rececy, x.frequency, x.monetary,
4  case
5  when rececy <= 180 and frequency >= 5 and monetary >= 300000 then 'VIP'
6  when rececy <= 365 and frequency >= 3 then 'loyal'
7  when rececy <= 700 then 'Lost Customer'
8  when frequency = 1 then 'One time buyer'
9  else 'regular'
10 end as segment from
11 (select customer_id,
12 datediff(current_date, max(oc.order_date)) as rececy,
13 count(distinct oc.order_id) as frequency,
14 sum(pc.amount) as monetary
15 from orders_c oc
16 inner join payments_c pc
17 on oc.order_id = pc.order_id
18 where oc.order_status = 'Delivered'
19 and pc.payment_status = 'success'
20 group by oc.customer_id) x;
```

	customer_id	rececy	frequency	monetary	segment
▶	C100000	689	3	219616	Lost Customer
	C100001	1172	1	80616	One time buyer
	C100002	C100001	3	120142	Lost Customer
	C100003	577	2	124003	Lost Customer
	C100004	821	4	181112	regular
	C100005	560	3	162968	Lost Customer

16) What are the top 5 selling products by quantity?

```
1  # What are the top 5 selling products by quantity?
2
3  • select pcc.product_name,
4          sum(oic.qty) as total_quantity
5          from products_c pcc
6          inner join order_items_c oic
7          on pcc.product_id = oic.product_id
8          group by pcc.product_name
9          order by total_quantity desc
10         limit 5;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
product_name	total_quantity		
Apple Item 473	362		
Apple Item 3675	357		
Mi Item 1333	347		
Mi Item 579	345		
Puma Item 4099	345		

17) What are the top-selling products by revenue?

```
1  # What are the top-selling products by revenue?
2
3  • select pcc.product_id,
4          pcc.product_name,
5          format(sum(oic.qty*oic.item_price),0) as total_revenue
6          from products_c pcc
7          inner join order_items_c oic
8          on pcc.product_id = oic.product_id
9          group by pcc.product_id, pcc.product_name
10         order by total_revenue desc
11         limit 5;
12
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
product_id	product_name	total_revenue		
P202005	Puma Item 2005	992,772		
P202354	Puma Item 2354	990,727		
P202897	Nike Item 2897	988,384		
P204042	Puma Item 4042	970,515		
P202567	Mi Item 2567	968,598		

18) What is the top product in each category?

```
1  # What is the top product in each category?
2
3  • select x.category,x.product_name,x.revenue from
4  (select
5      pcc.product_name,
6      pcc.category,
7      sum(oic.qty*oic.item_price) as revenue,
8      row_number() over(partition by pcc.category order by sum(oic.qty*oic.item_price) desc) as top_product
9      from products_c pcc
10     inner join order_items_c oic
11     on pcc.product_id = oic.product_id
12     inner join orders_c oc
13     on oc.order_id = oic.order_id
14     inner join payments_c pc
15     on oc.order_id = pc.order_id
16     where oc.order_status = 'delivered' and pc.payment_status = 'success'
17     group by pcc.product_name,pcc.category
18     order by revenue desc) x
19 where x.top_product = 1;
```

19) Identify products with low sales but high ratings.

```
1  # Identify products with low sales but high ratings.
2
3  • select pcc.product_name, sum(oic.qty*oic.item_price) as sales, round(avg(sc.rating),2) as rating
4      from products_c pcc
5      inner join order_items_c oic
6          on pcc.product_id = oic.product_id
7      inner join sellers_c sc
8          on oic.seller_id = sc.seller_id
9      group by pcc.product_name
10     order by sales asc, rating desc
11     limit 5;
12
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

	product_name	sales	rating
▶	Puma Item 2953	47200	3.96
	Apple Item 1182	53352	3.91
	Apple Item 3040	61659	3.88
	Lg Item 529	65786	3.89
	Puma Item 4069	85073	3.79

20) Which products have the highest return rate?

```
27  # Which products have the highest return rate?
28
29  •  select
30      p.product_name,
31      sum(oi.qty) as total_sold,
32      sum(case when r.order_id is not null then oi.qty else 0 end) as total_returned,
33      round(sum(case when r.order_id is not null then oi.qty else 0 end) / sum(oi.qty), 2) as return_rate
34  from products_c p
35  join order_items_c oi
36      on p.product_id = oi.product_id
37  left join returns_c r
38      on oi.order_id = r.order_id
39  group by p.product_name
40  order by return_rate desc
41  limit 5;
42
43
```

product_name	total_sold	total_returned	return_rate
Boat Item 3848	232	51	0.22
Puma Item 3747	216	43	0.20
Nike Item 2337	204	41	0.20
Puma Item 4847	241	47	0.20
Boat Item 74	259	48	0.19

21) Who are the top-performing sellers by revenue?

```
1  # Who are the top-performing sellers by revenue?
2
3  •  select
4      oic.seller_id,
5      sc.seller_name,
6      format(sum(oic.qty*oic.item_price),0) as total_sales
7  from order_items_c oic
8  inner join sellers_c sc
9      on oic.seller_id = sc.seller_id
10 group by oic.seller_id, sc.seller_name
11 order by total_sales
12 desc limit 5;
```

seller_id	seller_name	total_sales
S300386	Seller_386	40,142,448
S300303	Seller_303	40,111,041
S301074	Seller_1074	39,047,241
S301953	Seller_1953	39,019,020
S300899	Seller_899	38,818,794

22) Identify low-rated sellers with high sales.

```
1  # Identify low-rated sellers with high sales.
2
3  • select
4  oic.seller_id,sc.seller_name,
5  sum(oic.qty*oic.item_price) as sales,
6  avg(sc.rating) as avg_rating
7      from order_items_c oic
8      inner join sellers_c sc
9      on oic.seller_id = sc.seller_id
10     group by oic.seller_id,sc.seller_name
11     order by sales desc, avg_rating asc
12     limit 5;
```

Result Grid				
		Filter Rows:	Export: 	
		Wrap Cell Content: 		
	seller_id	seller_name	sales	avg_rating
▶	S300386	Seller_386	40142448	4
	S300303	Seller_303	40111041	4
	S301074	Seller_1074	39047241	3
	S301953	Seller_1953	39019020	4
	S300899	Seller_899	38818794	3

23) What is the average delivery time per seller?

```
1  # What is the average delivery time per seller?
2
3  • select
4  sc.seller_id,
5  seller_name,
6  avg(datediff(oc.delivery_date, oc.order_date)) as avg_delivery_days
7      from orders_c oc
8      inner join payments_c pc
9      on oc.order_id = pc.order_id
10     inner join order_items_c oic
11     on pc.order_id = oic.order_id
12     inner join sellers_c sc
13     on oic.seller_id = sc.seller_id
14     where oc.order_status = 'Delivered' and pc.payment_status = 'success'
15     group by sc.seller_id, seller_name
16     order by avg_delivery_days
17     limit 10;
```

24) What is the average delivery time by city?

```
1  # Q29. What is the average delivery time by city?
2
3  •  select sc.city,
4     avg(datediff(oc.delivery_date, oc.order_date)) as avg_delivery_days
5     from orders_c oc
6     inner join order_items_c oic
7         on oc.order_id = oic.order_id
8     inner join payments_c pc
9         on oic.order_id = pc.order_id
10    inner join sellers_c sc
11        on oic.seller_id = sc.seller_id
12    where oc.order_status = 'Delivered' and pc.payment_status = 'success'
13    group by sc.city
14    order by avg_delivery_days;
15
```

25) When was each seller's last completed order?

```
1  # When was each seller's last completed order?
2
3  •  select sc.seller_id,
4     sc.seller_name,
5     max(oc.order_date) as late_delivery_by_seller
6     from orders_c oc
7     inner join order_items_c oic
8         on oc.order_id = oic.order_id
9     inner join payments_c pc
10        on oic.order_id = pc.order_id
11    inner join sellers_c sc
12        on oic.seller_id = sc.seller_id
13    where oc.order_status = 'delivered' and pc.payment_status = 'success'
14    group by sc.seller_id, sc.seller_name;
```


26) What % of total revenue is refunded?

```
1  # What % of total revenue is refunded?
2
3  • select
4  concat(round(((x.refund/y.revenue)*100),2), ' ', '%') as refund_percentage
5      from
6  (select
7      sum(rc.refund_amount) as refund
8      from order_items_c oic
9      inner join returns_c rc
10     on oic.order_id = rc.order_id) x,
11
12  (select
13      sum(pc.amount) as revenue
14      from orders_c oc
15      inner join payments_c pc
16      on oc.order_id = pc.order_id
17      where oc.order_status = 'Delivered'
18      and pc.payment_status = 'success') y;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

refund_percentage
22.51 %

27) Payment mode success rate

```
1  # Payment mode success rate
2
3  • select
4  x.payment_status,
5  round(x.successful_payment/y.total_payment*100,2) as success_rate
6  from
7  (select payment_status, count(order_id) as successful_payment
8  from payments_c
9  group by payment_status
10  having payment_status = 'success') x,
11
12  (select count(payment_status) as total_payment from payments_c) y;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

payment_status	success_rate
Success	81.94

28) What is the impact of delivery delays on revenue?

On time within 6 days

After 6 days delayed

```
1  # What is the impact of delivery delays on revenue?
2  # On time within 6 days
3  # After 6 days delayed
4
5  • select
6  case
7      when dlc.delivered_time_hours > 144 then 'Delayed'
8      else 'On time'
9  end as delivery_status,
10 format(sum(pc.amount),0) as total_revenue
11     from orders_c oc
12     inner join payments_c pc
13         on oc.order_id = pc.order_id
14     inner join delivery_logs_c dlc
15         on oc.order_id = dlc.order_id
16     where oc.order_status = 'Delivered' and pc.payment_status = 'success'
17     group by delivery_status;
```

Result Grid

	delivery_status	total_revenue
▶	Delayed	1,599,980,765
	On time	5,496,320,570

29) Which sellers should be penalized?

```
1  # Which sellers should be penalized?
2
3  • select
4  oic.seller_id,
5  count(distinct oic.order_id) as total_orders,
6  count(distinct rc.order_id) as returns_,
7  round(count(distinct rc.order_id)/count(distinct oic.order_id)* 100,2) as return_rate
8      from order_items_c oic
9      left join returns_c rc
10         on oic.order_id = rc.order_id
11      group by oic.seller_id
12      having return_rate > 10;
```

Result Grid

	seller_id	total_orders	returns_	return_rate
▶	S300004	253	28	11.07
	S300012	261	29	11.11
	S300021	259	28	10.81
	S300025	269	31	11.52

30) Revenue split between new vs repeat customers (monthly)

```

1  # Revenue split between new vs repeat customers (monthly)
2
3  • select date_format(oc.order_date, '%Y-%M') as month_,
4     sum(pc.amount) as revenue,
5     case
6     when oc.order_date = fo.first_order_date then 'New'
7     else 'Repeat'
8     end as customer_type,
9     round(sum(pc.amount)/sum(sum(pc.amount)) over(partition by date_format(oc.order_date, '%Y-%M'))*100,2) as percentage
10    from orders_c oc inner join
11        (select
12         oc.customer_id, min(oc.order_date) as first_order_date
13         from orders_c oc
14         inner join payments_c pc
15         on oc.order_id = pc.order_id
16         where oc.order_status = 'delivered' and pc.payment_status = 'success'
17         group by oc.customer_id) fo
18    on oc.customer_id = fo.customer_id
19    inner join payments_c pc
20    on oc.order_id = pc.order_id
21    where oc.order_status = 'delivered' and pc.payment_status = 'success'
22    group by month_, customer_type
23    order by month_

```

	month_	revenue	customer_type	percentage
▶	2023-April	189866364	New	75.51
	2023-April	61570227	Repeat	24.49
	2023-August	122513370	Repeat	47.06
	2023-August	137832822	New	52.94
	2023-December	93406223	New	36.53
	2023-December	162311714	Repeat	63.47
	2023-February	27160866	Repeat	11.76
	2023-February	203786771	New	88.24
	2023-January	248454442	New	96.18
	2023-January	9867424	Repeat	3.82
	2023-July	148110668	New	57.25
	2023-July	110591421	Repeat	42.75
	2023-June	151766118	New	63.24
	2023-June	88232184	Repeat	36.76
	2023-March	205793167	New	81.63
	2023-March	46311642	Repeat	18.37
	2023-May	176929328	New	69.71
	2023-May	76883633	Repeat	30.29
	2023-November	110204393	New	42.52
	2023-November	148983226	Repeat	57.48
	2023-October	144911296	Repeat	55.35
	2023-October	116907870	New	44.65
	2023-September	125844313	New	49.80
	2023-September	126843562	Repeat	50.20
	2024-April	185446109	Repeat	72.71
	2024-April	69594151	New	27.29

31) Seller cancellation rate

```

1      # Seller cancellation rate
2
3      • select *,
4          round((coalesce(z.cancelled_orders, 0) * 100.0) / z.total_orders, 2) as cancellation_rate
5          from
6      (select x.seller_id, x.total_orders, y.cancelled_orders
7          from
8              (select oic.seller_id, count(oic.order_id) as total_orders
9                  from order_items_c oic
10                 inner join orders_c oc
11                     on oic.order_id = oc.order_id
12                 inner join payments_c pc
13                     on oc.order_id = pc.order_id
14                 group by oic.seller_id) x
15         left join
16             (select oic.seller_id, count(oic.order_id) as cancelled_orders
17                 from order_items_c oic
18                 inner join orders_c oc
19                     on oic.order_id = oc.order_id
20                 inner join payments_c pc on oc.order_id = pc.order_id
21                 where oc.order_status = 'Cancelled'
22                 group by oic.seller_id)y
23         on x.seller_id = y.seller_id)z

```

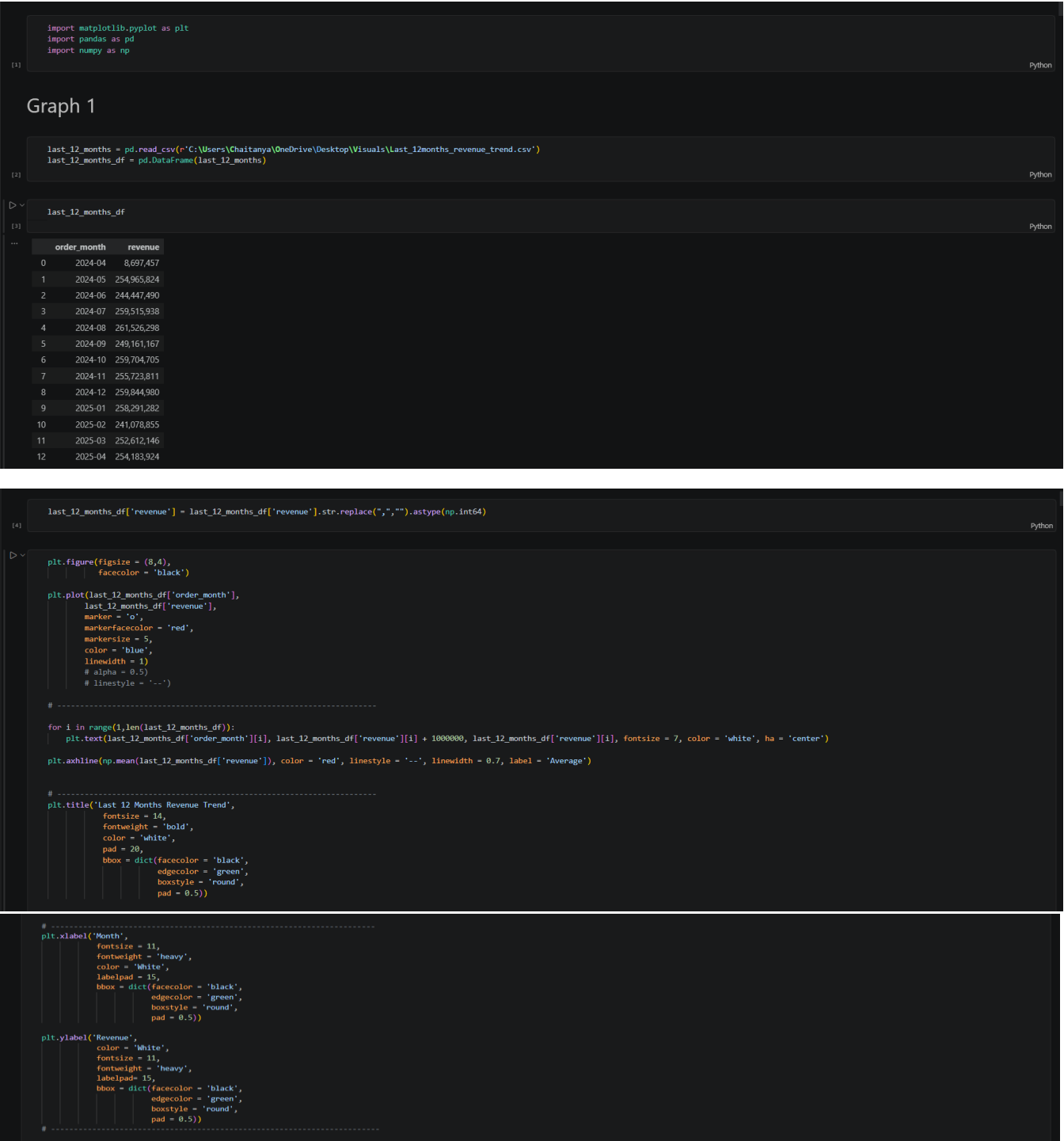
	seller_id	total_orders	cancelled_orders	cancellation_rate
▶	S301556	250	30	12.00
	S301282	240	27	11.25
	S300334	270	32	11.85
	S300685	239	19	7.95
	S300534	257	13	5.06
	S301674	240	20	8.33
	S300096	235	21	8.94
	S300237	219	20	9.13
	S301496	260	29	11.15
	S300428	260	28	10.77
	S300524	222	26	11.71
	S301599	253	21	8.30
	S301591	254	16	6.30
	S301906	235	22	9.36
	S300986	247	24	9.72
	S300160	249	22	8.84
	S301247	254	25	9.84
	S301810	257	25	9.73
	S301684	220	16	7.27
	S301878	234	13	5.56
	S301509	251	31	12.35
	S301781	253	23	9.09
	S301051	262	29	11.07
	S300333	248	25	10.08
	S301427	249	28	11.24
	S300858	266	23	8.65

Data Visualization and Insights

1. Monthly Revenue Trend

Objective - To analyze the monthly revenue trend over the last 12 months.

Python Code ↓



```
plt.xticks(fontsize = 9,
           rotation = 90,
           color = 'white',
           fontweight = 'normal')
plt.tick_params(axis = 'x', pad = 3)

plt.yticks(fontsize = 9,
           color = 'white',
           fontweight = 'normal')
plt.tick_params(axis = 'y', pad = 4)

plt.tick_params(axis = 'both', pad = 4)

# -----

plt.grid(axis = 'x', linestyle = ':', color = 'green')
plt.ylim(230000000, 265000000)

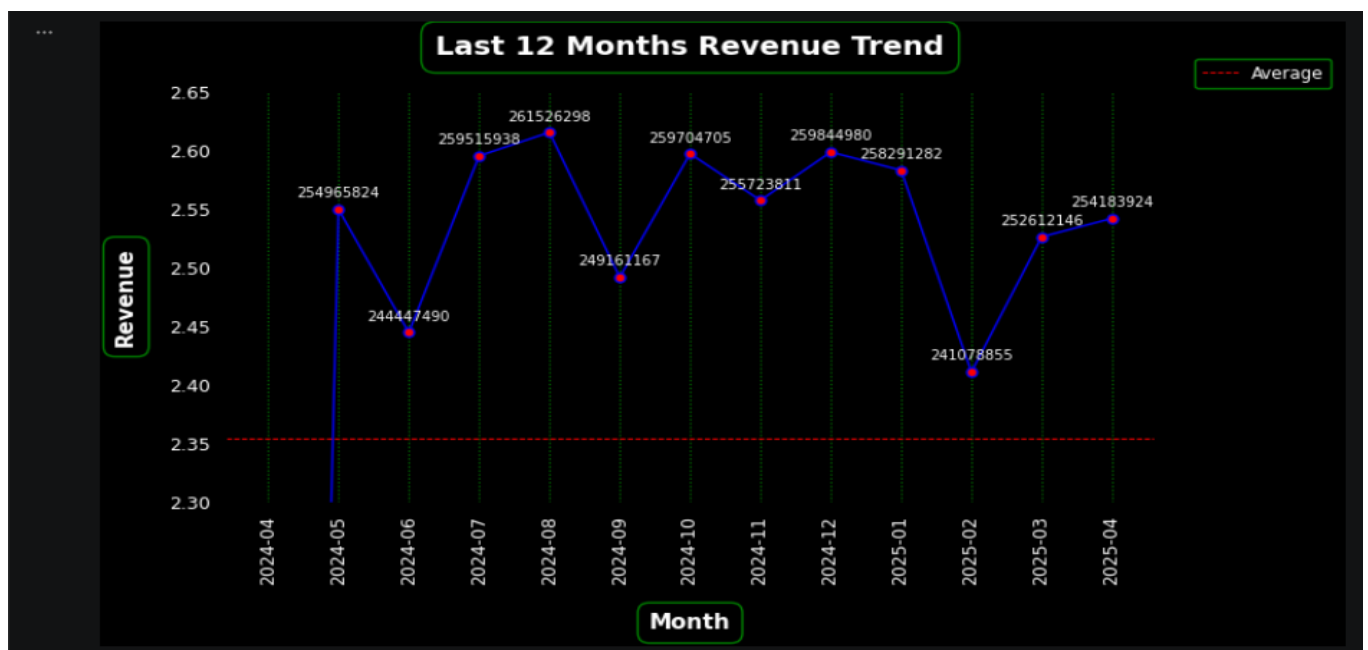
# -----

plt.legend(fontsize = 8.5,
           loc = 'best',
           bbox_to_anchor = (1.2,1.1),
           frameon=True,
           facecolor='black',
           edgecolor='green',
           framealpha=1)

legend = plt.gca().get_legend()
plt.setp(legend.get_texts(), color='white')

plt.gca().set_facecolor('black')
plt.gca().spines[['top','right']].set_visible(False)
# plt.gca().spines[['left','bottom']].set_color('red')

plt.show()
```



Interpretation

Revenue trends indicate the overall sales performance across months and help identify growth periods and seasonal fluctuations.

2. Month-over-Month Revenue Growth

Objective - To evaluate the percentage growth in revenue compared to the previous month.

Python Code 

Graph 2

```
data2 = pd.read_csv(r'C:\Users\Chaitanya\OneDrive\Desktop\Visuals\Month_over_Month_growth.csv')
month_over_month_growth_df = pd.DataFrame(data2)
```

[6] Python

```
month_over_month_growth_df
```

[7] Python

```
'''
order_month  current_month.revenue  previous_month.revenue  month_over_month.growth
0      2024-04          255040260             18654271          1267.19 %
1      2024-05          254965824             255040260           -0.03 %
2      2024-06          244447490             254965824          -4.13 %
3      2024-07          259515938             244447490           6.16 %
4      2024-08          261526298             259515938           0.77 %
5      2024-09          249161167             261526298          -4.73 %
6      2024-10          259704705             249161167           4.23 %
7      2024-11          255723811             259704705          -1.53 %
8      2024-12          259844980             255723811           1.61 %
9      2025-01          258291282             259844980          -0.60 %
10     2025-02          241078855             258291282          -6.66 %
11     2025-03          252612146             241078855           4.78 %
12     2025-04          254183924             252612146           0.62 %
'''
```

```
month_over_month_growth_df['month_over_month.growth'] = month_over_month_growth_df['month_over_month.growth'].str.replace('X', '').astype(float)
```

[8] Python

```
month_over_month_growth_df = month_over_month_growth_df.iloc[1:]
```

[9] Python

```
plt.figure(figsize = (8,4),
            facecolor = 'black')
colours = []

for values in month_over_month_growth_df['month_over_month.growth']:
    if values > 0:
        colours.append('green')
    else:
        colours.append('red')

plt.bar(month_over_month_growth_df['order_month'],
        month_over_month_growth_df['month_over_month.growth'],
        color = colours)

for i in range(1,len(month_over_month_growth_df) + 1):
    if month_over_month_growth_df['month_over_month.growth'][i] > 0:
        y_pos = month_over_month_growth_df['month_over_month.growth'][i] + 0.5
    else:
        y_pos = month_over_month_growth_df['month_over_month.growth'][i] - 0.5

    plt.text(month_over_month_growth_df['order_month'][i], y_pos,
            month_over_month_growth_df['month_over_month.growth'][i], fontsize = 7, color = 'white', ha = 'center')
```

```
# -----
plt.title('Month Over Month Growth',
        fontsize = 15,
        fontweight = 'bold',
        color = 'white',
        y = 1.1,
        bbox = dict(facecolor = 'black',
                    edgecolor = 'green',
                    boxstyle = 'round',
                    pad = 0.4))
```

```
plt.xlabel('Month',
        fontsize = 11,
        fontweight = 'heavy',
        color = 'white',
        labelpad = 20,
        bbox = dict(facecolor = 'black',
                    edgecolor = 'green',
                    boxstyle = 'round',
                    pad = 0.5))

plt.ylabel('Month over Month Growth',
        color = 'white',
        fontsize = 11,
        fontweight = 'heavy',
        labelpad = 15,
        bbox = dict(facecolor = 'black',
                    edgecolor = 'green',
                    boxstyle = 'round',
                    pad = 0.5))

# -----
plt.xticks(rotation = 90,
        color = 'white',
        fontweight = 'normal')
plt.tick_params(axis = 'x', pad = 5)

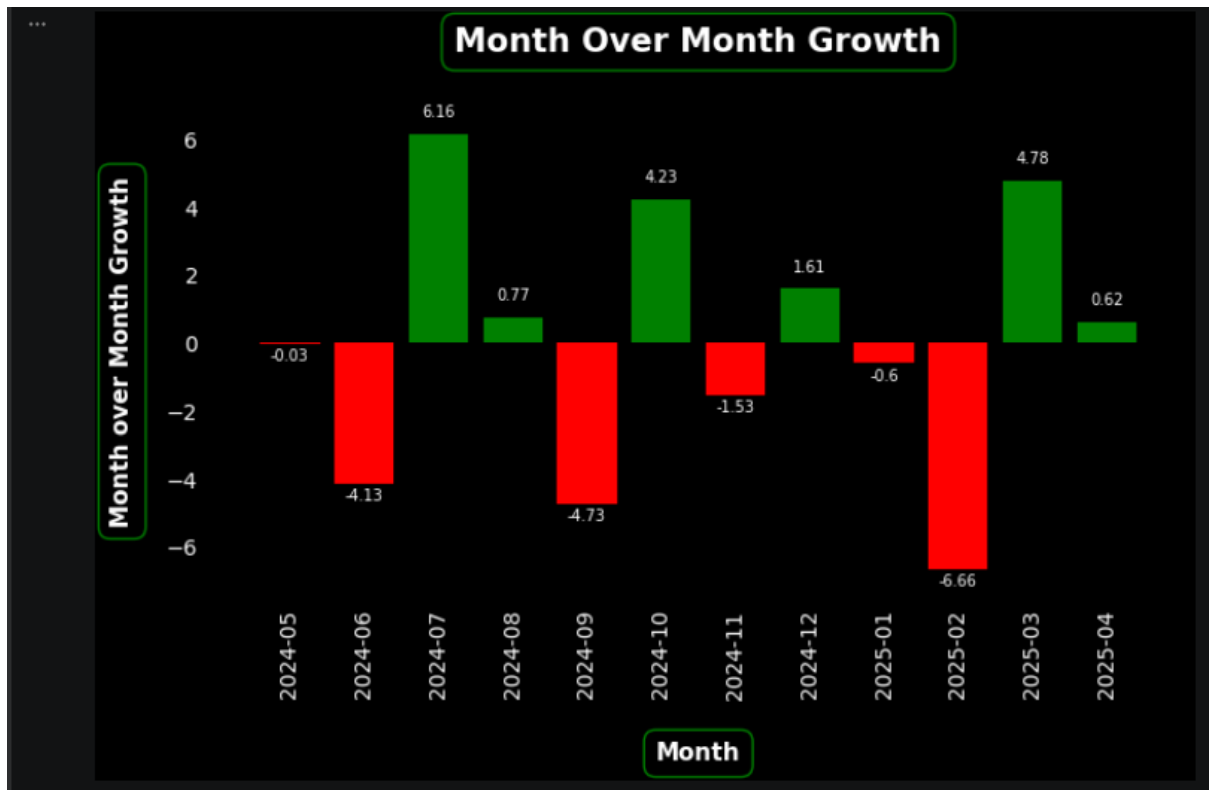
plt.yticks(color = 'white',
        fontweight = 'normal')
plt.tick_params(axis = 'y', pad = 5)

# -----
```

```
plt.gca().set_facecolor('black')
```

```
plt.show()
```

[10] Python



Interpretation

Positive growth indicates business expansion, while negative growth highlights periods requiring further investigation.

3. Top Revenue Generating Cities

Objective - To identify the cities contributing the highest revenue and understand the geographical distribution of sales.

Python Code



```
Graph 3

> ~
[11] data3 = pd.read_csv(r'C:\Users\Chaitanya\OneDrive\Desktop\Visuals\Top_10_Cities_by_rev.csv')
Python

[12] top_10_cities_by_rev = pd.DataFrame(data3)
Python

[13] top_10_cities_by_rev
'''
   city  total_revenue
0  Chennai  724,035,916
1  Ahmedabad  715,710,236
2  Lucknow  714,103,587
3  Mumbai  713,275,796
4  Kolkata  712,170,758
5  Hyderabad  710,853,139
6  Pune  710,680,328
7  Delhi  703,418,066
8  Jaipur  699,998,353
9  Bengaluru  692,055,156

[14] top_10_cities_by_rev['total_revenue'] = top_10_cities_by_rev['total_revenue'].str.replace(",","").astype(int)
Python
```

```
import random
from matplotlib.ticker import FuncFormatter

(15)

plt.figure(figsize = (8,4), facecolor = 'black')
colour = []
for _ in range(len(top_10_cities_by_rev)):
    colour.append((
        random.random(),
        random.random(),
        random.random()
    ))

plt.barh(top_10_cities_by_rev['city'],top_10_cities_by_rev['total_revenue'],color = colour)

plt.xlim(690000000, 725000000) # Axis zoomin

for i in range(len(top_10_cities_by_rev['total_revenue'])):
    plt.text(top_10_cities_by_rev['total_revenue'][i] + 50000, top_10_cities_by_rev['city'][i], top_10_cities_by_rev['total_revenue'][i],va= 'center' ,color = 'white',fontsize = 8)

plt.title('Top 10 Cities by Revenue',
        fontsize = 15,
        fontweight = 'bold',
        color = 'white',
        pad = 20,
        bbox = dict(
            facecolor = 'black',
            edgecolor = 'green',
            boxstyle = 'round',
            pad = 0.4))

plt.xlabel('Total Revenue',
        fontsize = 11,
        fontweight = 'heavy',
        color = 'white',
        labelpad = 20,
        bbox = dict(facecolor = 'black',
            edgecolor = 'green',
            boxstyle = 'round',
            pad = 0.5))
```

```
plt.ylabel('City',
        color = 'white',
        fontsize = 11,
        fontweight = 'heavy',
        labelpad = 15,
        bbox = dict(facecolor = 'black',
            edgecolor = 'green',
            boxstyle = 'round',
            pad = 0.5))

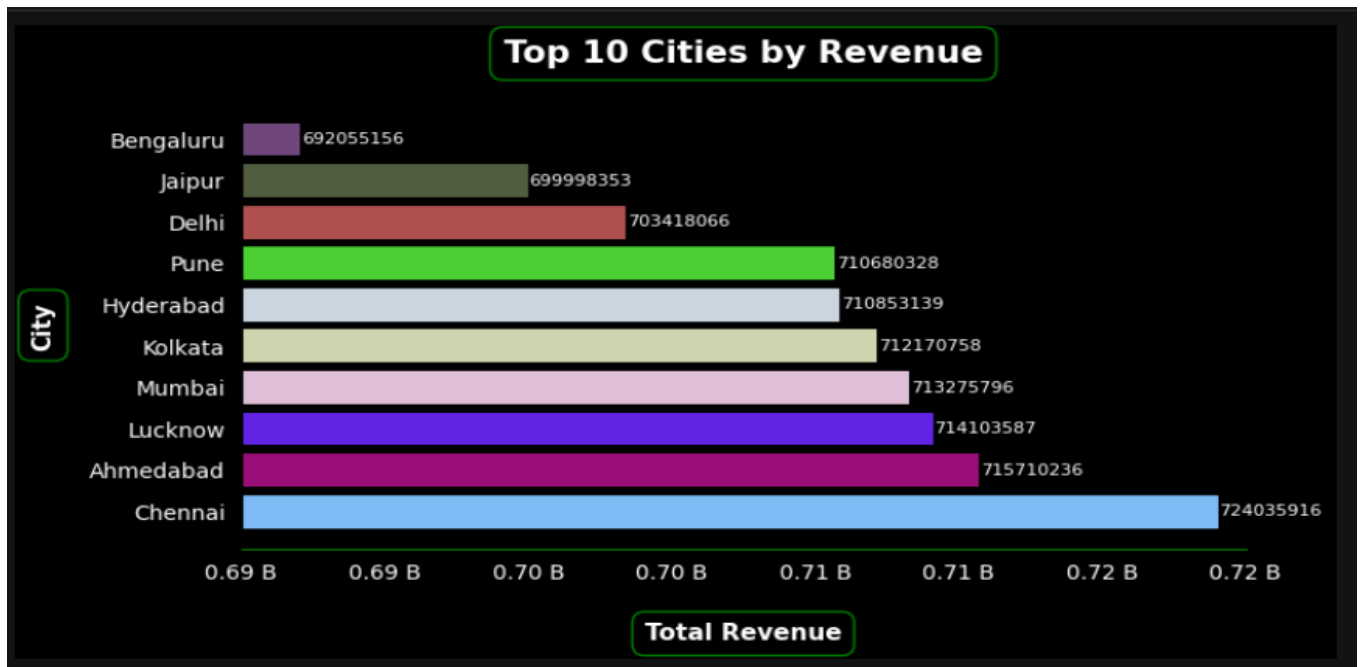
plt.xticks(color = 'white',
        fontweight = 'normal')
#rotation = 90

plt.yticks(color = 'white',
        fontweight = 'normal')

plt.gca().xaxis.set_major_formatter(
    FuncFormatter(lambda x, pos: f'{x/1e9:.2f} B')
)

plt.gca().spines[['top','right']].set_visible(False)
plt.gca().spines[['bottom']].set_color('green')
plt.gca().set_facecolor('black')

plt.show()
```



Interpretation

The visualization highlights the top revenue-generating cities. A significant portion of total revenue is concentrated in a few cities, indicating strong market demand and customer activity in those regions. These cities can be prioritized for targeted marketing campaigns, inventory planning, and business expansion strategies to maximize revenue growth.

4. Average Order Value (AOV) by City

Objective - To analyze the average order value across different cities and identify locations where customers spend more per order.

Python Code 

Graph 4

```
[17] data4 = pd.read_csv(r'C:\Users\Chaitanya\OneDrive\Desktop\Visuals\avg_order_value_by_city.csv') Python

[18] avg_order_value_by_city_df = pd.DataFrame(data4) Python

[19] avg_order_value_by_city_df Python

...
   city  total_revenue  AOV_Average_order_value
0  Chennai      724035916      60051.08
1  Ahmedabad    715710236      60042.81
2  Lucknow     714103587      59933.16
3  Mumbai     713275796      60263.25
4  Kolkata    712170758      60348.34
5  Hyderabad  710853139      59540.43
6  Pune       710680328      60242.46
7  Delhi      703418066      60327.45
8  Jaipur     699998353      60443.69
9  Bengaluru  692055156      60136.87

[20] avg_order_value_by_city_df['total_revenue'] = avg_order_value_by_city_df['total_revenue'].astype(int)
avg_order_value_by_city_df['AOV_Average_order_value'] = avg_order_value_by_city_df['AOV_Average_order_value'].astype(float) Python
```

```
plt.figure(figsize = (8,4), facecolor = 'black')

colour1 = []
for _ in range(len(avg_order_value_by_city_df)):
    colour1.append((random.random(),
                    random.random(),
                    random.random()))

plt.bar(avg_order_value_by_city_df['city'], avg_order_value_by_city_df['AOV_Average_order_value'],color = colour1)
plt.ylim(59000, 60600)

for i in range(len(avg_order_value_by_city_df)):
    plt.text(avg_order_value_by_city_df['city'][i],
             avg_order_value_by_city_df['AOV_Average_order_value'][i] + 20,
             avg_order_value_by_city_df['AOV_Average_order_value'][i],
             ha = 'center',
             fontsize = 8,
             color = 'white')

plt.title('Average Order Value By City',
         fontsize = 15,
         fontweight = 'bold',
         color = 'white',
         pad = 20,
         bbox = dict(
             facecolor = 'black',
             edgecolor = 'green',
             boxstyle = 'round',
             pad = 0.4))

plt.xlabel('City',
         fontsize = 11,
         fontweight = 'heavy',
         color = 'white',
         labelpad = 20,
         bbox = dict(facecolor = 'black',
             edgecolor = 'green',
             boxstyle = 'round',
             pad = 0.5))
```



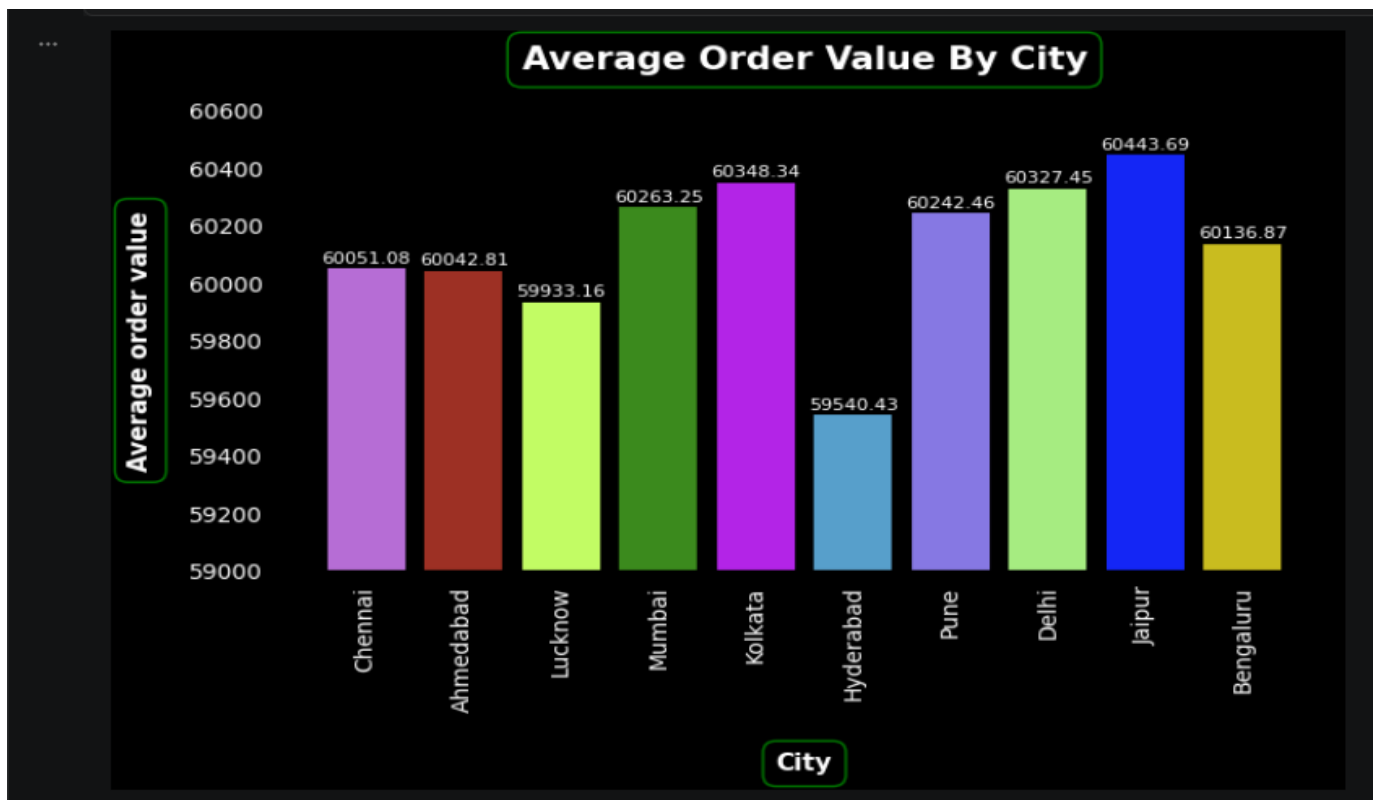
```
plt.ylabel("Average order value",
          color = 'white',
          fontsize = 11,
          fontweight = 'heavy',
          labelpad= 15,
          bbox = dict(facecolor = 'black',
                      edgecolor = 'green',
                      boxstyle = 'round',
                      pad = 0.5))

plt.yticks(color = 'white',
           fontweight = 'normal')

plt.xticks(rotation = 90,
           fontweight = 'normal',
           color = 'white')

plt.gca().spines[['top','right','left']].set_visible(False)
plt.gca().spines[['left','bottom']].set_color('black')
plt.gca().set_facecolor('black')
# plt.gca().set_axisbelow(True)

# plt.grid(axis = 'y', linestyle = '--', linewidth = 0.5, color = 'green' )
plt.show()
```



Interpretation

The visualization compares the average order value across major cities. Cities with higher AOV indicate stronger customer purchasing power and higher-value transactions. These locations present opportunities for premium product offerings, targeted promotions, and customer retention initiatives. Understanding AOV by city helps the business optimize pricing strategies and improve revenue generation.

5. Revenue by Product Category

Objective - To identify the product categories generating the highest revenue and understand their contribution to overall business performance.

Graph 5

```
[22] data5 = pd.read_csv(r'C:\Users\Chaitanya\OneDrive\Desktop\Visuals\Category_wise_Revenue_Performance.csv') Python

[23] Category_wise_Revenue_Performance_df = pd.DataFrame(data5) Python

[24] Category_wise_Revenue_Performance_df Python

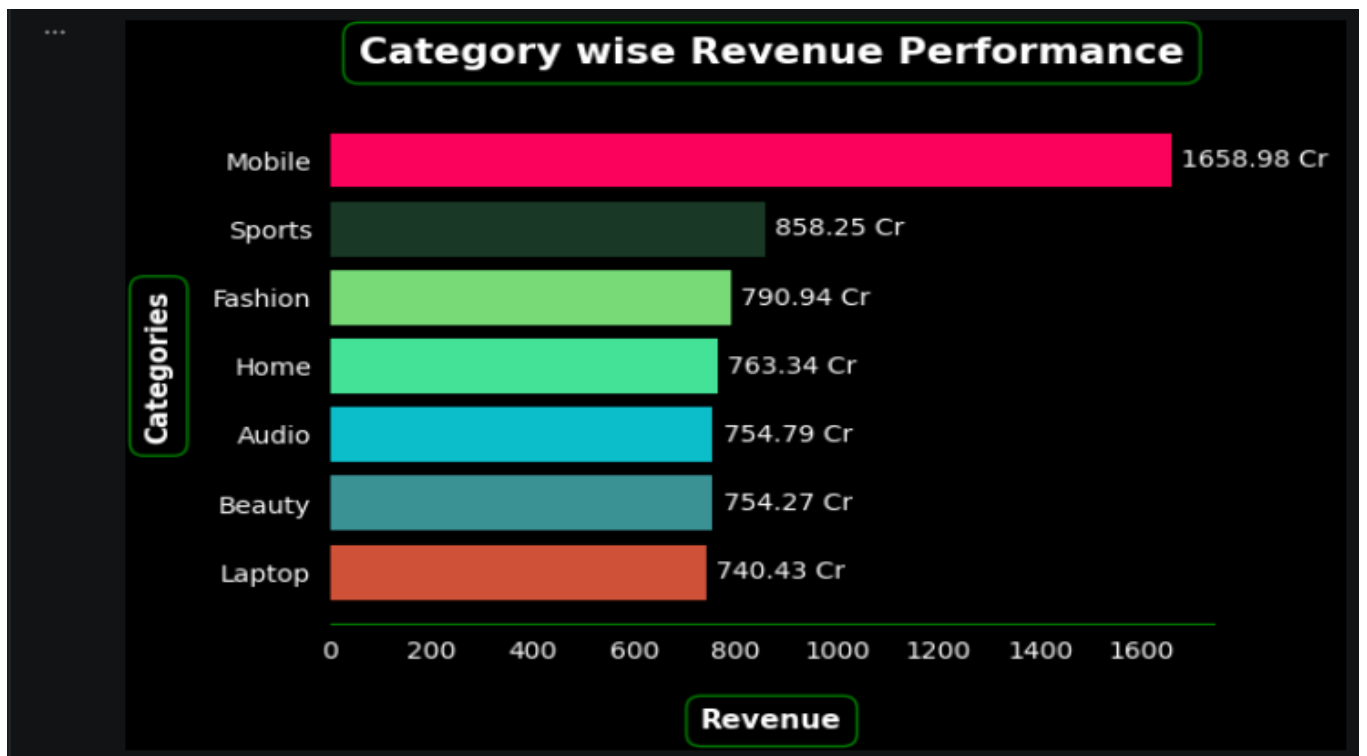
'''
category    total_revenue
0    Laptop    7,404,274,410
1    Beauty    7,542,694,002
2    Audio     7,547,949,776
3    Home      7,633,441,231
4    Fashion   7,909,351,670
5    Sports    8,582,520,400
6    Mobile    16,589,789,813

Category_wise_Revenue_Performance_df['total_revenue'] = Category_wise_Revenue_Performance_df['total_revenue'] .str.replace(",","").astype(int) Python

import random Python
```

```
[25] plt.figure(figsize = (6,4),
      |         facecolor = 'black')
      | Category_wise_Revenue_Performance_df['revenue_cr'] = (Category_wise_Revenue_Performance_df['total_revenue'] / 1e7).round(2)
      | colour = []
      | for _ in range(len(Category_wise_Revenue_Performance_df)):
      |     colour.append((random.random(),
      |                   random.random(),
      |                   random.random()))
      |
      | plt.barh(Category_wise_Revenue_Performance_df['category'],Category_wise_Revenue_Performance_df['revenue_cr'], color = colour)
      |
      | for i in range(len(Category_wise_Revenue_Performance_df)):
      |     plt.text((Category_wise_Revenue_Performance_df['revenue_cr'])[i] + 20,
      |              Category_wise_Revenue_Performance_df['category'][i],
      |              f'{Category_wise_Revenue_Performance_df["revenue_cr"][i]} Cr',
      |              va = 'center',
      |              color = 'white')
      |
      | plt.title('Category wise Revenue Performance',
      |          |         fontsize = 15,
      |          |         fontweight = 'bold',
      |          |         color = 'white',
      |          |         pad = 20,
      |          |         bbox = dict(
      |          |             facecolor = 'black',
      |          |             edgecolor = 'green',
      |          |             boxstyle = 'round',
      |          |             pad = 0.4))
      |
      | plt.xlabel('Revenue',
      |          |         fontsize = 11,
      |          |         fontweight = 'heavy',
      |          |         color = 'white',
      |          |         labelpad = 20,
      |          |         bbox = dict(facecolor = 'black',
      |          |             edgecolor = 'green',
      |          |             boxstyle = 'round',
      |          |             pad = 0.5))
```

```
      | plt.ylabel('Categories',
      |          |         color = 'white',
      |          |         fontsize = 11,
      |          |         fontweight = 'heavy',
      |          |         labelpad = 15,
      |          |         bbox = dict(facecolor = 'black',
      |          |             edgecolor = 'green',
      |          |             boxstyle = 'round',
      |          |             pad = 0.5))
      |
      | plt.gca().spines[['top','right','left']].set_visible(False)
      | plt.gca().set_facecolor('black')
      |
      | plt.xticks(color = 'white')
      | plt.yticks(color = 'white')
      |
      | # plt.grid(axis = 'x', linestyle = '--', color = 'green') #, linewidth = 0.5)
      |
      | plt.gca().set_axisbelow(True)
      | plt.gca().spines['bottom'].set_color('Green')
      |
      | plt.show() Python
```



Interpretation

The visualization highlights revenue generated by different product categories. Categories contributing the highest revenue represent the company's strongest product segments and play a crucial role in overall business growth. Lower-performing categories may require additional marketing efforts, pricing adjustments, or product portfolio optimization to improve sales performance.

6. Customer Purchase Frequency Analysis (Repeat vs One-Time Customers)

Objective - To analyze customer purchasing behavior by comparing repeat customers and one-time customers.

Python Code 

```

Graph 6

[20] data6 = pd.read_csv(r"C:\Users\Chaitanya\OneDrive\Desktop\Visuals\Repeat_oneTime_customer.csv")
Python

customer_purchase_frequency = pd.DataFrame(data6)
print(customer_purchase_frequency['repeat_customers'].astype)
print(customer_purchase_frequency['one_time_customers'].astype)
Python

<bound method NDFrame.astype of 0 45405
Name: repeat_customers, dtype: int64>
<bound method NDFrame.astype of 0 3681
Name: one_time_customers, dtype: int64>

[21] customer_purchase_frequency
Python

repeat_customers  one_time_customers
0                45405             3681

```

```

plt.figure(figsize = (5,5), facecolor = 'black')

labels = ['Repeat Customers', 'One Time Buyers']

size = [
    customer_purchase_frequency['repeat_customers'][0],
    customer_purchase_frequency['one_time_customers'][0],
]

color = ['cyan', 'gold']
wedges, texts, autotexts = plt.pie(size, labels = labels, autopct = '%1.1%', textprops={'color':'white',
                                                    'fontSize':10,
                                                    'fontWeight':'heavy'}, labeldistance= 1.1, pctdistance = 0.6, colors = color)

# Donut hole
centre_circle = plt.Circle(
    (0, 0),
    0.80,
    fc='black'
)

plt.title('Customer Purchase Behavior Distribution',
          fontsize = 15,
          color = 'white',
          fontweight = 'bold',
          bbox = dict(facecolor = 'black',
                      edgecolor = 'green',
                      boxstyle = 'round',
                      pad = 0.4))

for i, text in enumerate(autotexts):
    text.set_color(color[i])
    text.set_fontsize(11)
    text.set_fontweight('bold')

table_data = [
    ['Repeat Customers', '45,405'],
    ['One-Time Customers', '3,681']
]

```

```

# -----
plt.figtext(
    0.01,
    0.03,

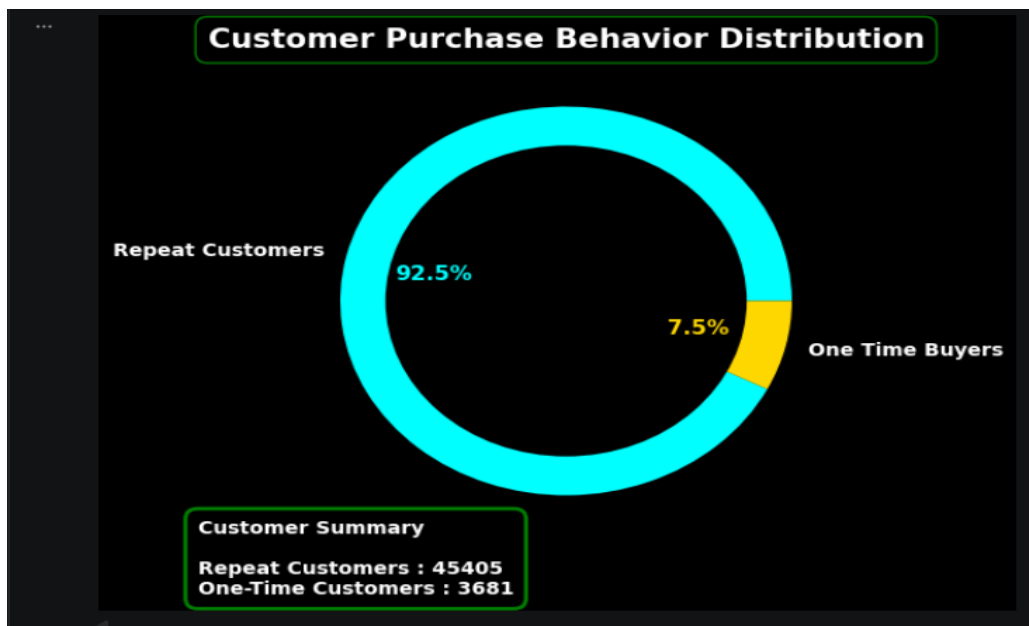
    f"Customer Summary\n\n"
    f"Repeat Customers : {customer_purchase_frequency['repeat_customers'][0]}\n"
    f"One-Time Customers : {customer_purchase_frequency['one_time_customers'][0]}",

    color='white',
    fontsize=10,
    fontweight='bold',

    bbox=dict(
        facecolor='black',
        edgecolor='green',
        boxstyle='round,pad=0.6',
        linewidth=2
    )
)

plt.gca().add_artist(centre_circle)
plt.show()

```



Interpretation

The donut chart illustrates the distribution of repeat and one-time customers. A higher proportion of repeat customers indicates strong customer satisfaction, brand loyalty, and effective retention strategies. Repeat customers are generally more valuable as they contribute to consistent revenue generation and lower customer acquisition costs.

7. Customer Segmentation using RFM Analysis

Objective - To classify customers into different segments based on their purchasing behavior using Recency, Frequency, and Monetary (RFM) analysis.

Python Code 

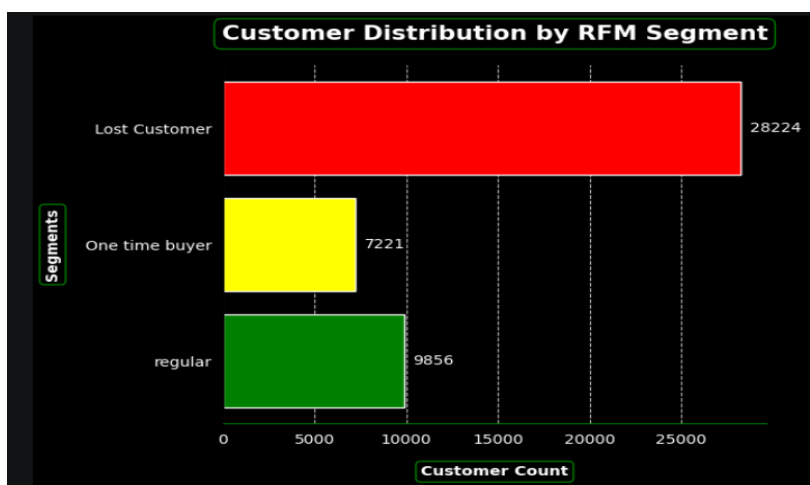
Graph 7

```
[33] data7 = pd.read_csv(r'C:\Users\Chaitanya\OneDrive\Desktop\Visuals\RFM segmentation1.csv')  
Python  
[34] RFM_Segmentation_df = pd.DataFrame(data7)  
Python  
[35] RFM_Segmentation_df  
Python  
...
```

	segment	rfm
0	regular	9856
1	One time buyer	7221
2	Lost Customer	28224

```
plt.title('Customer Distribution by RFM Segment',  
         color = 'white',  
         fontweight = 'bold',  
         fontsize = 15,  
         pad = 20,  
         bbox = dict(edgecolor = 'green',  
                     pad = 0.3,  
                     boxstyle = 'round',  
                     facecolor = 'black'))  
  
plt.xlabel('Customer Count',  
          color = 'white',  
          fontweight = 'heavy',  
          fontsize = 10,  
          labelpad = 15,  
          bbox = dict(edgecolor = 'green',  
                      pad = 0.3,  
                      boxstyle = 'round',  
                      facecolor = 'black'))  
  
plt.ylabel('Segments',  
         color = 'white',  
         fontweight = 'heavy',  
         fontsize = 10,  
         labelpad = 15,  
         bbox = dict(edgecolor = 'green',  
                     pad = 0.3,  
                     boxstyle = 'round',  
                     facecolor = 'black'))
```

```
(function) axis: Literal['both', 'x', 'y']  
plt.grid(axis = 'x', linestyle = '--')  
plt.gca().set_axisbelow(True)  
  
plt.xticks(color = 'white')  
plt.yticks(color = 'white')  
plt.gca().set_facecolor('black')  
plt.gca().spines[['top', 'right']].set_visible(False)  
plt.gca().spines[['bottom']].set_color('green')  
  
plt.show()  
[36]
```

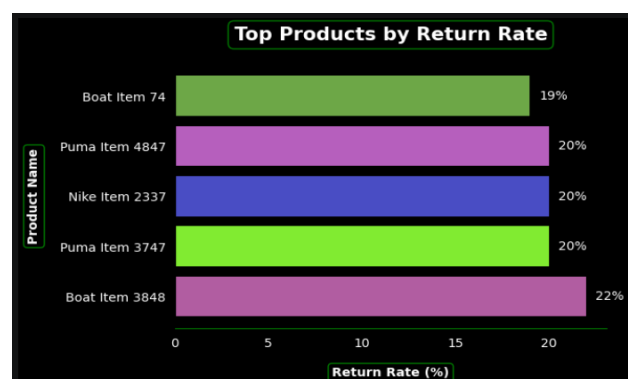
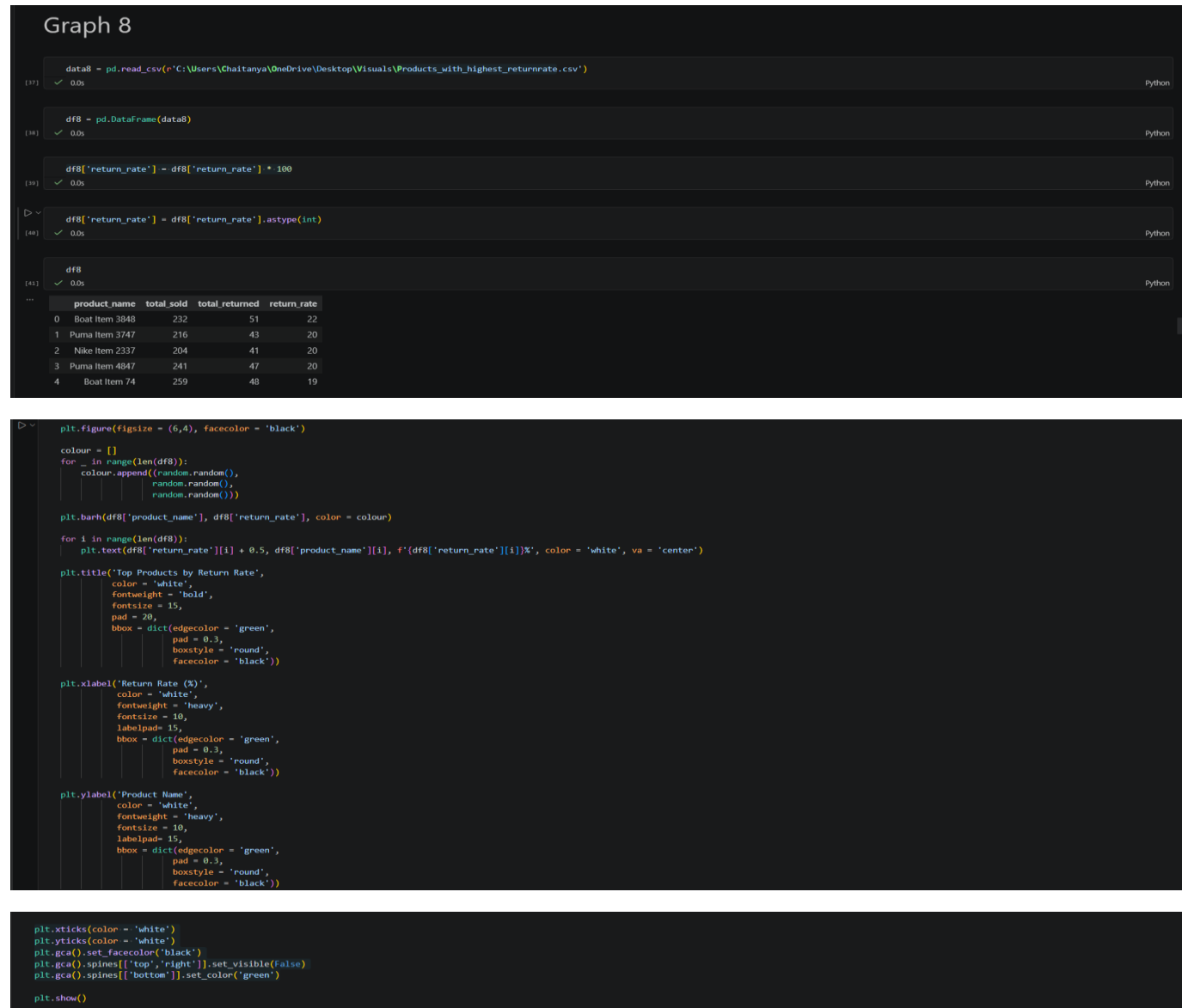


Interpretation- The RFM segmentation analysis reveals that the majority of customers belong to the **Lost Customer** segment, indicating that a significant number of customers have not made recent purchases and may require re-engagement efforts. The **Regular Customer** segment represents active customers who continue to generate revenue for the business. The **One-Time Buyer** segment consists of customers who have made only a single purchase and may need targeted retention strategies to encourage repeat transactions. Overall, the analysis highlights opportunities to improve customer retention and strengthen long-term customer relationships.

8. Products with Highest Return Rates

Objective - To identify products with the highest return rates and evaluate potential product quality, customer satisfaction, or fulfillment issues.

Python Code 



Interpretation- The visualization highlights the products with the highest return rates. A high return rate may indicate issues related to product quality, customer expectations, inaccurate product descriptions, or delivery conditions. Analyzing return-prone products can help the business identify operational challenges and improve customer satisfaction by reducing product returns.

9. Revenue Impact of Delivery Delays

Objective - To compare the revenue generated from on-time and delayed deliveries and assess the impact of delivery performance on business revenue.

Graph 9

```
[44] ✓ 0.0s Python
data9 = pd.read_csv(r'C:\Users\Chaitanya\OneDrive\Desktop\Visuals\Impact_of_delivery_delays_on_revenue.csv')

[45] ✓ 0.0s Python
df9 = pd.DataFrame(data9)

[46] ✓ 0.0s Python
df9

```

	delivery_status	total_revenue
0	Delayed	1,599,980,765
1	On time	5,496,320,570

```

[47] ✓ 0.0s Python
df9['total_revenue'] = df9['total_revenue'].str.replace(',','').astype(int)

[48] ✓ 0.0s Python
df9['revenue_billion'] = (df9['total_revenue'] / 1000000000)

```

```

plt.figure(figsize = (6,4), facecolor = 'black')
plt.bar(df9['delivery_status'], df9['revenue_billion'], color = ['orange','green'])

for i in range(len(df9)):
    plt.text(df9['delivery_status'][i], df9['revenue_billion'][i] + 0.2, f'{df9["revenue_billion"][i]:.2f} billion',
             color = 'white', ha = 'center')

plt.title("Revenue Impact of Delivery Delays",
          color = 'white',
          fontweight = 'bold',
          fontsize = 15,
          pad = 20,
          bbox = dict(edgecolor = 'green',
                     pad = 0.3,
                     boxstyle = 'round',
                     facecolor = 'black'))

plt.xlabel('Delivery Status',
           color = 'white',
           fontweight = 'heavy',
           fontsize = 10,
           labelpad = 15,
           bbox = dict(edgecolor = 'green',
                      pad = 0.3,
                      boxstyle = 'round',
                      facecolor = 'black'))

plt.ylabel('Revenue (Billions)',
           color = 'white',
           fontweight = 'heavy',
           fontsize = 10,
           labelpad = 15,
           bbox = dict(edgecolor = 'green',
                      pad = 0.3,
                      boxstyle = 'round',
                      facecolor = 'black'))

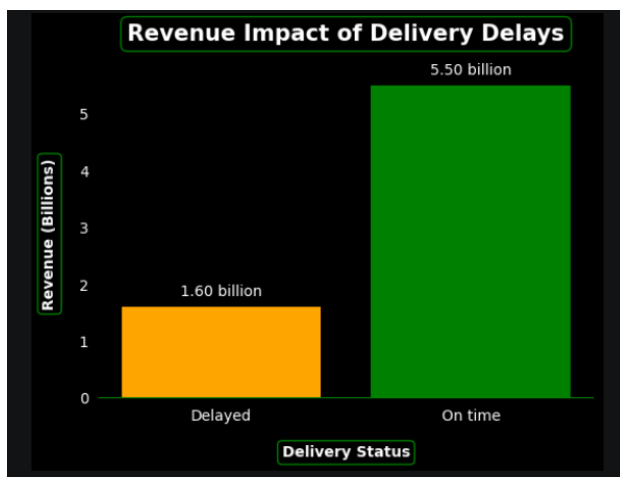
```

```

plt.xticks(color = 'white')
plt.yticks(color = 'white')
plt.gca().set_facecolor('black')
plt.gca().spines[['top', 'right']].set_visible(False)
plt.gca().spines[['bottom']].set_color('green')

plt.show()
[49] ✓ 0.0s Python

```



Interpretation

The visualization compares revenue generated from orders delivered on time and those delivered with delays. The results indicate that on-time deliveries contribute significantly more revenue than delayed deliveries. This highlights the importance of efficient logistics and timely order fulfillment in maintaining customer satisfaction and supporting revenue growth. Consistent delivery performance can positively influence customer trust, repeat purchases, and overall business performance.

10) Monthly Revenue Contribution by Customer Type

Objective - To analyze the monthly revenue contribution of new and repeat customers and evaluate customer retention trends over time.

Python Code



Graph 10

```
data10 = pd.read_csv(r'C:\Users\Chaitanya\OneDrive\Desktop\Visuals\revenue_split_new_vs_repeat_customers.csv')
df10 = pd.DataFrame(data10)

stacked_df10 = df10.pivot(
    index='month_',
    columns='customer_type',
    values='revenue'
)
print(stacked_df10)
```

customer_type	New	Repeat
month		
2023-April	189866364	61570227
2023-August	137832822	122513370
2023-December	93480223	162311714
2023-February	203786771	27168866
2023-January	248454442	9867424
2023-July	148110668	110591421
2023-June	151766118	88232184
2023-March	205793167	46311642

```
plt.figure(figsize = (12,5),facecolor = 'black')

plt.bar(stacked_df10.index,
        stacked_df10['New'],
        label = 'New Customers',
        color = 'green' )

plt.bar(stacked_df10.index,
        stacked_df10['Repeat'],
        label = 'Repeat customers',
        bottom = stacked_df10['New'],
        color = 'orange' )

for i in range(len(stacked_df10)):
    total = (stacked_df10['New'].iloc[i] + stacked_df10['Repeat'].iloc[i])
    plt.text(stacked_df10.index[i], total + 1995000, f'{total/1000000:.0f} Cr', color = 'white', ha = 'center', fontsize = 7)

plt.title('Monthly Revenue Contribution: New vs Repeat Customers',
        color = 'white',
        fontweight = 'bold',
        fontsize = 15,
        pad = 30,
        bbox = dict(edgecolor = 'green',
                    pad = 0.3,
                    boxstyle = 'round',
                    facecolor = 'black'))

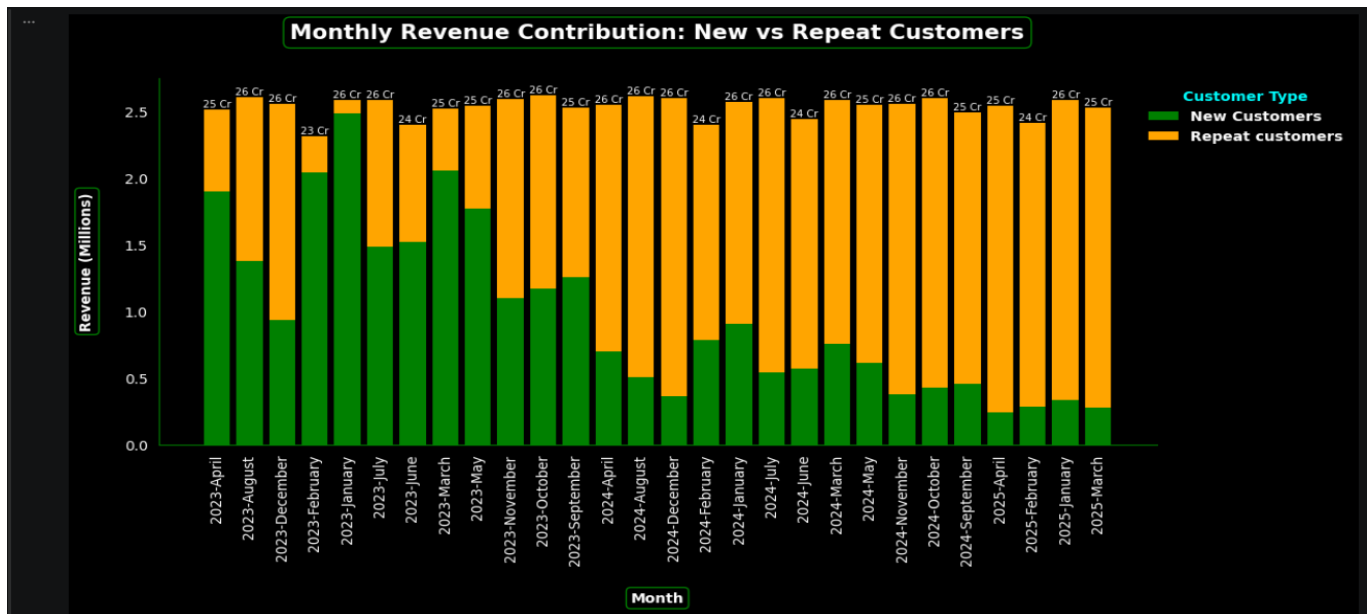
plt.xlabel('Month',
        color = 'white',
        fontweight = 'heavy',
        fontsize = 10,
        labelpad = 20,
        bbox = dict(edgecolor = 'green',
                    pad = 0.3,
                    boxstyle = 'round',
                    facecolor = 'black'))
```

```
plt.ylabel('Revenue (Millions)',
        color = 'white',
        fontweight = 'heavy',
        fontsize = 10,
        labelpad = 20,
        bbox = dict(edgecolor = 'green',
                    pad = 0.3,
                    boxstyle = 'round',
                    facecolor = 'black'))

plt.xticks(color = 'white')
plt.yticks(color = 'white')
plt.xticks(rotation = 90)
plt.gca().set_facecolor('black')
plt.gca().spines[['top', 'right']].set_visible(False)
plt.gca().spines[['left', 'bottom']].set_color('green')

legend = plt.legend(title = 'Customer Type',
                    loc = 'upper right',
                    bbox_to_anchor = (1.20,1),
                    facecolor = 'black',
                    edgecolor = 'black')
for text in legend.get_texts():
    text.set_color('white')
    text.set_fontweight('heavy')

legend.get_title().set_color('cyan')
legend.get_title().set_fontweight('bold')
plt.show()
```

Interpretation

The stacked bar chart presents the monthly revenue contribution from new and repeat customers. The analysis shows that repeat customers contribute a substantial share of total revenue across most months, highlighting the importance of customer retention in driving business performance. While new customers continue to support business growth, repeat customers generate consistent revenue and play a critical role in long-term profitability. The findings indicate that maintaining customer loyalty is essential for sustainable revenue growth.

6. Key Findings and Insights

1. Revenue showed a positive growth trend over the analysis period.

The monthly revenue analysis revealed steady business growth, with several months recording higher sales compared to previous periods. This indicates increasing customer demand and business expansion.

2. A few cities contributed a significant share of total revenue.

Revenue distribution analysis showed that sales were concentrated in a limited number of cities. These locations represent key markets and should be prioritized for future business growth initiatives.

3. Customer retention plays a major role in revenue generation.

The customer purchase frequency analysis revealed that repeat customers contribute a substantial portion of total revenue, highlighting the importance of customer loyalty and retention strategies.

4. Lost Customers represent the largest customer segment.

RFM segmentation identified Lost Customers as the largest group within the customer base. This suggests that a significant number of customers are not making repeat purchases and may require targeted re-engagement efforts.

5. Product returns remain a critical operational concern.

The return rate analysis highlighted specific products with relatively high return percentages. These products may require further investigation related to product quality, customer expectations, or product descriptions.

6. On-time deliveries contribute significantly higher revenue than delayed deliveries.

The delivery performance analysis demonstrated that orders delivered on time generated substantially more revenue than delayed orders. This emphasizes the importance of efficient logistics and order fulfillment processes.

7. Repeat customer revenue increased over time.

The monthly customer type analysis showed that repeat customers contributed a growing share of total revenue across multiple months, indicating improved customer retention and long-term customer value.

8. Data-driven analysis helps identify growth opportunities.

By combining SQL-based analysis with Python visualizations, the project successfully identified revenue drivers, customer behaviour patterns, operational challenges, and opportunities for business improvement.

7. Recommendations

1. Implement Customer Retention Campaigns

Since Lost Customers represent the largest customer segment, the company should introduce targeted retention campaigns, personalized offers, and re-engagement strategies to encourage repeat purchases.

2. Strengthen Customer Loyalty Programs

The analysis showed that repeat customers contribute a significant portion of total revenue. Loyalty programs, reward points, and exclusive benefits can help increase customer retention and lifetime value.

3. Focus on High-Revenue Cities

Cities generating the highest revenue should be prioritized for marketing investments, inventory planning, and customer acquisition efforts to maximize business growth.

4. Reduce Product Return Rates

Products with high return rates should be reviewed to identify potential issues related to product quality, packaging, product descriptions, or customer expectations. Reducing returns can improve profitability and customer satisfaction.

5. Improve Delivery Performance

Since on-time deliveries contribute significantly more revenue than delayed deliveries, the company should focus on optimizing logistics operations and delivery processes to enhance customer experience.

6. Promote High-Performing Product Categories

The business should increase promotional activities and inventory availability for high-revenue product categories while evaluating strategies to improve the performance of lower-revenue categories.

7. Use Data Analytics for Decision-Making

Regular monitoring of sales performance, customer behaviour, and operational metrics through data analytics can help management make informed business decisions and identify future growth opportunities.

Conclusion

This project focused on analyzing e-commerce sales data using SQL and Python to gain insights into business performance, customer behaviour, and operational efficiency. Various analytical techniques were applied to evaluate revenue trends, customer purchase patterns, product performance, delivery performance, and customer segmentation.

The analysis revealed that customer retention plays a crucial role in revenue generation, while delivery performance and product return significantly impact overall business outcomes. RFM segmentation helped identify different customer groups, enabling a deeper understanding of customer behaviour and retention opportunities.

The visualizations and insights generated through this project demonstrate how data-driven decision-making can support business growth, improve customer satisfaction, and optimize operational performance. Overall, the project highlights the practical application of SQL, Pandas, and Matplotlib in solving real-world business problems and generating actionable business insights.